

UNIVERSITATEA “LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE ȘI INGINERIE ELECTRICĂ

PROIECT DE DIPLOMĂ

Îndrumător: Asist. drd. ing. Precup Ștefan Alexandru

Coordonator: Prof. dr. ing. Gellert Arpad

Absolvent

Dicoi Darius-Dumitru

Specializarea

Calculatoare

UNIVERSITATEA “LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE ȘI INGINERIE ELECTRICĂ

Aplicație web pentru promovarea obiectivelor turistice

Îndrumător: Asist. drd. ing. Precup Ștefan Alexandru

Coordonator: Prof. dr. ing. Gellert Arpad

Absolvent

Dicoi Darius-Dumitru

Specializarea

Calculatoare

Cuprins:

1.Introducere	1
1.1 Scopul și importanța lucrării.....	1
1.2 Domeniul de aplicabilitate	2
2. Considerații teoretice	3
2.1 Turismul digital și impactul tehnologiei.....	3
2.2 Analiza soluțiilor existente și surse de inspirație.....	5
2.3 Metode și tehnologii utilizate	7
2.3.1 Frontend -VueJs	9
2.3.2 Backend – C# .NET API	13
2.3.3 PostgreSQL și PostGIS	16
2.3.4 LM Studio și modele LLM.....	20
3. Rezolvarea temei de proiect	23
3.1 Definirea cerințelor prin user stories și planificarea funcționalităților.....	23
3.2 Modelarea bazei de date.....	27
3.2.1 Crearea bazei de date inițiale	27
3.2.2 Conectarea backend-ului la PostgreSQL.....	28
3.2.3 Comunicarea dintre C# și PostgreSQL	28
3.2.4 Rolul Entity Framework Core în gestionarea datelor	29
3.2.5 Sistemul de migrări	30
3.2.6 Instalarea extensiei PostGIS	31
3.2.7 Utilizarea proprietății Point în entități	31
3.2.8 Calcularea distanțelor dintre utilizator și entități.....	32
3.3 Implementarea backend-ului în ASP.NET Core	33
3.3.1 Definirea entității User.....	34
3.3.2 Implementarea procesului de înregistrare.....	36
3.3.3 Configurarea JWT în fișierul appsettings.json	37
3.3.4 Generarea Bearer Token-ului.....	37
3.3.5 Importanța Bearer Token-ului în sesiunea utilizatorului	39
3.3.6 Implementarea principalelor entități ale sistemului	39
3.3.7 Relația dintre Objective și ObjectiveType	40
3.3.8 Relația dintre Objective și ObjectiveImage	41
3.3.9 Relația dintre Objective și Review	41
3.3.10 Implementarea entităților Event, Itinerary și ItineraryDetails	42

3.3.11 Implementarea operațiilor CRUD	43
3.3.12 Menținerea fiabilității codului	47
3.4 Implementarea frontend-ului în Vue.js	48
3.4.1 Configurarea proiectului cu Vite.....	49
3.4.2 Implementarea paginii de autentificare	49
3.4.3 Implementarea paginii pentru afișarea obiectivelor turistice	50
3.4.4 Modulul de administrare.....	52
3.4.5 Implementarea paginii de detaliu pentru obiectiv	53
3.4.6 Dezvoltarea celorlalte pagini	55
3.5 Algoritm de recomandare.....	55
3.5.1 Încarcarea obiectivelor	56
3.5.2 Normalizarea datelor	58
3.5.3 Determinarea numărului optim de clustere	60
3.5.4 Aplicarea algoritmului K-Means și atribuirea clusterelor	63
3.5.5 Persistența rezultatelor clusterizării	67
3.5.6 Generarea recomandărilor pe baza clusterelor obținute	69
3.6 Proiectarea ChatBot-ului	72
3.6.1 Construirea bazei de date vectoriale.....	73
3.6.2 Segmentarea documentelor.....	75
3.6.3 Generarea embedding-urilor și construirea bazei vectoriale	75
3.6.4 Căutarea semantică a informațiilor relevante	77
3.6.5 Construirea promptului și generarea răspunsului	78
3.6.6 Integrarea modelului lingvistic local prin LM Studio.....	80
3.6.7 Expunerea chatbotului printr-un serviciu API.....	81
4. Concluzii și dezvoltări ulterioare	82
4.1 Obiectivele îndeplinite	82
4.2 Posibile îmbunătățiri și dezvoltări ulterioare	84
4.3 Concluzii	85
Anexa 1.....	86
Bibliografie.....	87

1.Introducere

1.1 Scopul și importanța lucrării

Lucrarea de față are ca punct central dezvoltarea unei aplicații web moderne, denumită TravelS (Travel Service), concepută pentru a îmbunătăți accesul la informațiile turistice din România printr-o platformă digitală unitară, intuitivă și ușor de utilizat. Inițiativa acestui proiect a pornit dintr-o experiență personală și dintr-o nevoie reală resimțită în timpul propriilor călătorii. Fiind pasionat de descoperirea locurilor frumoase din țară, am constatat dificultatea identificării rapide și centralizate a obiectivelor turistice, a evenimentelor locale sau a activităților specifice unei anumite zone. De cele mai multe ori, informațiile sunt dispersate pe site-uri diferite, neactualizate sau greu de găsit, ceea ce îngreunează procesul de planificare al unei excursii sau vizite. Din dorința de a răspunde acestei provocări și de a contribui concret la digitalizarea turismului autohton, a luat naștere ideea dezvoltării platformei TravelS.

Scopul aplicației este de a sprijini turiștii, români sau străini, în identificarea facilă a celor mai relevante puncte de interes, experiențe și evenimente dintr-o anumită localitate din România, oferind în același timp o experiență digitală corectă, eficientă și plăcută. Utilizatorii nu mai sunt nevoiți să caute informații împrăștiate, ci pot accesa într-un singur loc conținut esențial pentru planificarea călătoriilor, explorarea obiectivelor sau alegerea celor mai potrivite activități pentru timpul lor liber. Importanța acestei lucrări nu rezidă doar în valoarea ei practică, ci și în potențialul de a contribui la promovarea patrimoniului cultural și natural al României, încurajând vizitarea unor locuri mai puțin cunoscute și sprijinind astfel turismul local.

Totodată, dezvoltarea acestei aplicații implică o abordare complexă, atât din punct de vedere tehnic, cât și logic și funcțional. Proiectul presupune nu doar agregarea și afișarea de date relevante, ci și organizarea acestora într-un mod inteligent, precum și integrarea unor componente moderne de interacțiune, printre care se numără și un chatbot care poate răspunde la întrebările utilizatorilor pe baza unui set de documente turistice procesate anterior. Această funcționalitate, deși aflată într-un stadiu incipient, are potențialul de a deveni un element definitoriu pentru asistența digitală oferită vizitatorilor.

Lucrarea este, astfel, relevantă nu doar prin aplicația practică imediată, ci și prin demonstrarea unei serii de concepte esențiale în dezvoltarea de soluții web inteligente, interconectate și orientate spre utilizator, într-un domeniu care are nevoie de inovație și digitalizare continuă.

1.2 Domeniul de aplicabilitate

TravelS se poziționează în domeniul aplicațiilor web dedicate ghidajului turistic digital, răspunzând nevoii tot mai evidente de a integra tehnologia în promovarea și accesarea informațiilor de interes cultural, istoric sau recreativ. Această platformă se adresează în principal turiștilor care intenționează să viziteze orașe și regiuni din România și care își doresc o soluție rapidă și eficientă pentru a afla ce pot face sau vedea într-un anumit loc. Fie că este vorba despre un vizitator ocazional sau despre cineva care explorează în mod regulat destinații noi, aplicația oferă o interfață care facilitează accesul la date esențiale fără efort suplimentar din partea utilizatorului.

Publicul-țintă include atât cetățeni români, cât și turiști străini interesați de cultura locală, de peisajele naturale sau de evenimentele specifice fiecărei zone. Aplicația poate deveni și un instrument util pentru autoritățile locale, pentru centrele de informare turistică sau pentru agențiile de turism care doresc să își promoveze mai eficient oferta, folosindu-se de o platformă digitală.

Funcționalitatea aplicației este concentrată pe prezentarea obiectivelor turistice într-un mod interactiv, susținut de posibilități avansate de filtrare și căutare, astfel încât fiecare utilizator să poată personaliza experiența în funcție de interese, locație sau tip de activitate dorită. În plus, integrarea chatbotului, care oferă suport prin extragerea de informații din documente turistice procesate anterior, contribuie la transformarea platformei într-un ghid digital accesibil și ușor de utilizat, fără a înlocui însă factorul uman sau recomandările personalizate bazate pe profilul utilizatorului, cel puțin nu în această etapă a dezvoltării.

Aplicația are un potențial de extindere considerabil. În viitor, pot fi dezvoltate funcționalități adiționale, precum generarea automată de itinerarii în funcție de interesele utilizatorului, integrarea cu servicii externe prin API-uri oficiale sau implementarea unor mecanisme de inteligență artificială care să analizeze comportamentele de căutare pentru a oferi sugestii cât mai relevante. TravelS nu este doar un punct de plecare pentru digitalizarea turismului local, ci și o platformă cu deschidere spre evoluție tehnologică și adaptabilitate, gândită pentru a oferi sprijin real în găsirea experiențelor din România.

2. Considerații teoretice

2.1 Turismul digital și impactul tehnologiei

Turismul reprezintă unul dintre cele mai dinamice sectoare ale economiei globale, fiind influențat în mod direct de transformările tehnologice și sociale produse în ultimele decenii. Dezvoltarea accelerată a tehnologiilor informaționale și extinderea accesului la internet au condus la apariția conceptului de turism digital, care presupune integrarea soluțiilor informatice în toate etapele experienței turistice, de la procesul de informare și planificare, până la consumul efectiv al serviciilor și evaluarea experienței după finalizarea călătoriei. Așa cum evidențiază literatura de specialitate, digitalizarea turismului a modificat fundamental relația dintre utilizatori și furnizorii de servicii, facilitând accesul rapid la informații, automatizarea proceselor și dezvoltarea unor servicii mai eficiente și mai personalizate [1].

Transformarea digitală a industriei turistice a produs schimbări semnificative și la nivelul comportamentului utilizatorilor. Dacă în trecut deciziile privind alegerea unei destinații sau a unui serviciu turistic erau influențate în principal de agențiile de turism, recomandările directe sau materialele tipărite, în prezent utilizatorii au acces la volume mari de informații disponibile online. Platformele digitale permit consultarea recenziilor, compararea ofertelor și analiza experiențelor altor turiști, ceea ce conduce la creșterea gradului de autonomie în procesul decizional. Motoarele de căutare, platformele sociale și website-urile specializate au devenit surse principale de informare pentru utilizatorii moderni, influențând direct alegerile acestora [2].

Un efect important al digitalizării este creșterea transparenței și competitivității în industria turistică. Platformele online permit compararea rapidă a ofertelor în funcție de preț, locație, facilități sau evaluările altor utilizatori, determinând furnizorii să își optimizeze constant serviciile și să adopte tehnologii moderne pentru a rămâne competitivi. În același timp, accesul la infrastructuri digitale a redus barierele de intrare pe piață, facilitând apariția unor platforme inovatoare și a unor noi modele de business bazate pe servicii digitale.

Impactul tehnologiei poate fi observat și la nivel operațional, prin automatizarea unor procese esențiale. Sistemele informatice moderne permit gestionarea rezervărilor, administrarea eficientă a resurselor și analiza comportamentului utilizatorilor. În industria hotelieră, sistemele de management integrat contribuie la optimizarea gradului de ocupare și la eficientizarea operațiunilor interne, în timp ce companiile aeriene utilizează algoritmi pentru ajustarea dinamică a prețurilor în funcție de cerere, sezonabilitate sau comportamentul pieței.

Un alt aspect definitoriu al turismului digital îl reprezintă personalizarea experienței utilizatorului. În contextul actual, turiștii nu mai caută exclusiv oferte standardizate, ci experiențe adaptate propriilor preferințe, bugete și interese. Colectarea și analiza datelor despre comportamentul utilizatorilor permit dezvoltarea unor sisteme inteligente capabile să genereze recomandări relevante. Sistemele de recomandare utilizează informații precum istoricul căutărilor, locația utilizatorului sau preferințele exprimate pentru a sugera obiective turistice, activități și itinerarii personalizate. Importanța acestor sisteme și impactul lor asupra experienței utilizatorului sunt evidențiate în cadrul lucrării „Recommender Systems Handbook”, unde este prezentat modul în care algoritmi de recomandare contribuie la creșterea gradului de satisfacție și la îmbunătățirea interacțiunii utilizatorului cu platformele digitale [3].

Inteligența artificială reprezintă una dintre cele mai importante direcții de dezvoltare în cadrul turismului digital, contribuind la automatizarea interacțiunii dintre utilizatori și platformele turistice. Integrarea tehnologiilor bazate pe inteligență artificială permite furnizarea unor servicii inteligente, capabile să proceseze volume mari de date și să răspundă rapid nevoilor utilizatorilor. Printre cele mai utilizate aplicații se numără chatboții și asistenții virtuali, care oferă suport în timp real pentru căutarea informațiilor, recomandarea destinațiilor sau planificarea itinerariilor. Studiile recente arată că utilizarea acestor tehnologii contribuie la reducerea timpului de răspuns, creșterea accesibilității informației și îmbunătățirea experienței utilizatorilor [4].

Chatboții reprezintă o componentă importantă a ecosistemului digital modern, fiind capabili să simuleze conversații umane și să ofere asistență automatizată într-un mod intuitiv. Prin utilizarea tehnicilor de procesare a limbajului natural și a modelelor de învățare automată, aceștia pot interpreta intențiile utilizatorilor și adapta răspunsurile în funcție de context. În turism, aceste sisteme permit furnizarea rapidă a informațiilor despre destinații, obiective turistice sau servicii disponibile, reducând dependența de intervenția umană și optimizând experiența generală.

Pe lângă inteligența artificială, alte tehnologii emergente precum realitatea augmentată și realitatea virtuală contribuie la transformarea experienței turistice. Realitatea virtuală oferă utilizatorilor posibilitatea explorării unor locații înainte de vizită, reducând incertitudinea asociată procesului decizional și influențând alegerea destinației. În paralel, realitatea augmentată îmbogățește experiența în timpul vizitei prin suprapunerea de informații digitale peste mediul real, facilitând accesul la informații despre obiective sau orientarea în spațiu.

Geolocalizarea și tehnologiile bazate pe date spațiale au devenit, de asemenea, componente esențiale în dezvoltarea aplicațiilor turistice moderne. Integrarea sistemelor GPS și a platformelor cartografice permite furnizarea de informații contextuale adaptate poziției utilizatorului. Astfel, utilizatorii pot identifica obiective turistice aflate în proximitate, pot calcula trasee și pot primi sugestii relevante în timp real, contribuind la o experiență mai interactivă și personalizată.

2.2 Analiza soluțiilor existente și surse de inspirație

În etapa de analiză și proiectare a aplicației TravelS a fost realizat un studiu asupra platformelor turistice existente, cu scopul identificării funcționalităților relevante și a modului în care acestea contribuie la îmbunătățirea experienței utilizatorului. Analiza soluțiilor deja disponibile pe piață a reprezentat un pas important în definirea structurii aplicației și în conturarea direcției generale de dezvoltare.

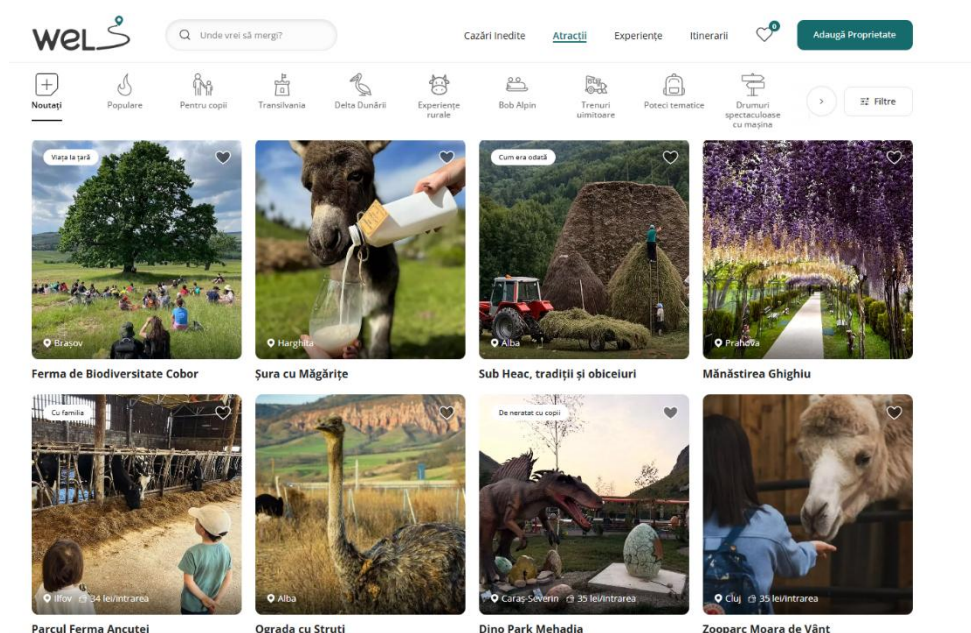


Figura 1 - Platforma wel.ro

Una dintre principalele surse de inspirație utilizate în cadrul acestui proiect a fost platforma wel.ro (Figura 1), o aplicație web orientată către promovarea cazărilor și experiențelor turistice din România. Platforma oferă o interfață modernă și intuitivă, organizată

în jurul unor categorii tematice care facilitează descoperirea rapidă a locațiilor și activităților de interes pentru utilizatori.

Un aspect important observat în cadrul acestei platforme este organizarea conținutului pe categorii specifice, precum cabane A-frame, locații pentru familii, experiențe de grup sau cazări cu facilități speciale. Această abordare contribuie la simplificarea procesului de căutare și permite utilizatorilor să identifice rapid opțiuni relevante în funcție de preferințele personale. În plus, platforma include secțiuni dedicate atracțiilor turistice, experiențelor și itinerariilor, oferind o experiență digitală integrată și orientată către planificarea completă a unei călătorii.

De asemenea, platforma pune la dispoziția utilizatorilor funcționalități de filtrare și organizare a conținutului, permițând restrângerea rezultatelor în funcție de diferite criterii. Acest mod de structurare a informației a reprezentat un element important de inspirație în proiectarea aplicației TravelS, în special pentru dezvoltarea sistemului de filtrare a obiectivelor turistice și pentru organizarea categoriilor disponibile în interfață.

Pe lângă componenta funcțională, analiza platformei a evidențiat și importanța designului vizual. Utilizarea imaginilor de mari dimensiuni, accentul pus pe prezentarea experiențelor și organizarea clară a informațiilor contribuie semnificativ la crearea unei experiențe interactive pentru utilizator.

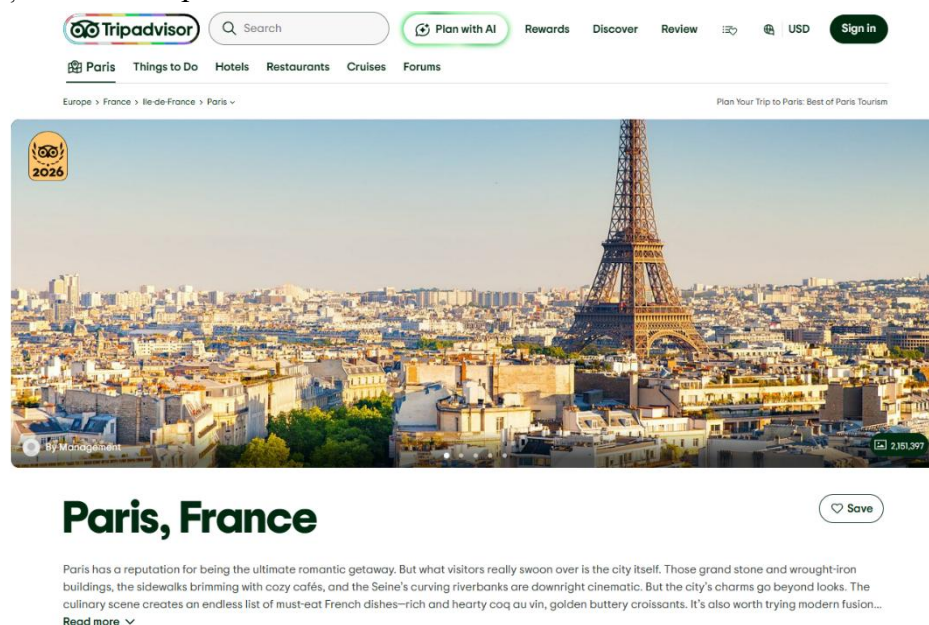


Figura 2 - Tripadvisor

O altă soluție existentă analizată în cadrul proiectului este platforma Tripadvisor (Figura 2), una dintre cele mai cunoscute aplicații internaționale dedicate planificării călătoriilor. Platforma permite utilizatorilor să caute informații despre destinații, obiective turistice,

hoteluri, restaurante și activități, fiind susținută de un volum foarte mare de recenzii și contribuții din partea comunității de călători. Tripadvisor pune accent pe experiențele reale ale utilizatorilor, oferind evaluări, comentarii, fotografii și recomandări care pot influența procesul de alegere a unei destinații sau a unei activități turistice. Platforma se prezintă ca una dintre cele mai mari comunități de călătorie, având peste un miliard de recenzii și opinii publicate de utilizatori.

Un aspect important observat în cadrul acestei platforme este modul în care informațiile sunt organizate în funcție de interesele utilizatorului. Tripadvisor oferă categorii precum obiective turistice, restaurante, hoteluri și activități, ceea ce permite utilizatorilor să își planifice mai ușor călătoria într-un singur ecosistem digital. De asemenea, existența recenziilor și a ratingurilor contribuie la creșterea încrederii în informațiile afișate, deoarece utilizatorii pot lua decizii pe baza experiențelor altor turiști. Această abordare a reprezentat o sursă de inspirație pentru aplicația TravelS în ceea ce privește integrarea recenziilor, afișarea ratingurilor și organizarea obiectivelor turistice într-un mod ușor de parcurs.

Comparativ cu Tripadvisor, aplicația TravelS are o direcție mai restrânsă și mai adaptată turismului din România. În timp ce Tripadvisor este o platformă globală, orientată către multiple tipuri de servicii turistice, TravelS urmărește centralizarea informațiilor despre obiective turistice, evenimente și experiențe locale din România. Prin această abordare, aplicația își propune să ofere un conținut mai specific și mai apropiat de nevoile utilizatorilor interesați de explorarea destinațiilor românești. În plus, TravelS integrează funcționalități proprii, precum recomandările bazate pe algoritmi de clustering, utilizarea datelor geospațiale pentru filtrarea obiectivelor apropiate și integrarea unui chatbot inteligent bazat pe documente turistice procesate anterior.

În cadrul procesului de dezvoltare, analiza soluțiilor existente nu a avut rolul de a reproduce funcționalitățile deja implementate, ci de a identifica bune practici și direcții eficiente de organizare a conținutului turistic. Astfel, platforma TravelS a fost proiectată ca o soluție proprie, adaptată obiectivelor aplicației și integrată cu funcționalități suplimentare, precum recomandările bazate pe algoritmi de clustering, integrarea chatbotului inteligent și utilizarea datelor geospațiale pentru afișarea obiectivelor turistice.

2.3 Metode și tehnologii utilizate

În cadrul dezvoltării aplicației TravelS a fost necesară alegerea unui set de tehnologii moderne, capabile să susțină cerințele unei platforme web interactive, orientate spre gestionarea și afișarea informațiilor turistice într-un mod eficient și scalabil. Alegerea tehnologiilor utilizate, prezente în Figura 3, s-a realizat ținând cont de mai multe aspecte importante, precum performanța aplicației, modularitatea componentelor, ușurința procesului de dezvoltare, posibilitatea extinderii ulterioare și compatibilitatea dintre diferitele tehnologii folosite în cadrul proiectului.

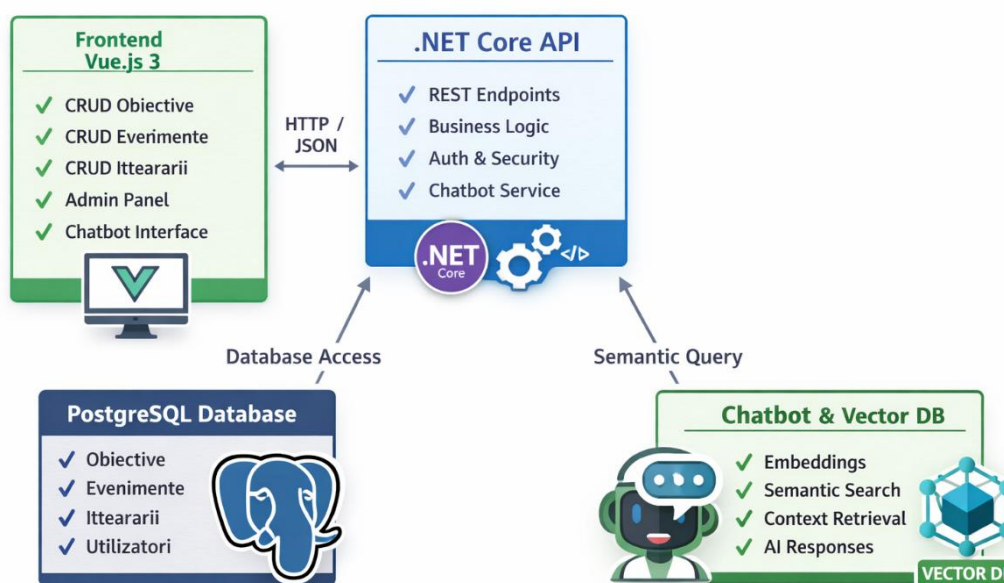


Figura 3 – Arhitectura aplicației

Arhitectura aplicației a fost construită pe baza unei separări clare între partea de interfață utilizator, backend și sistemul de gestionare a datelor, fiecare dintre acestea având un rol bine definit în cadrul funcționării aplicației. O astfel de abordare permite dezvoltarea independentă a componentelor, facilitează mentenanța aplicației și contribuie la obținerea unei structuri moderne și ușor de extins.

Pentru realizarea interfeței grafice a fost utilizat framework-ul Vue.js, alături de tehnologii și biblioteci complementare dedicate dezvoltării frontend. Alegerea acestora a avut rolul de a oferi o experiență fluidă utilizatorului, un timp redus de încărcare și posibilitatea construirii unei interfețe reactive și interactive.

Partea de backend a fost dezvoltată utilizând platforma C# ASP.NET Web API, o tehnologie robustă și performantă, utilizată frecvent pentru dezvoltarea serviciilor RESTful și a aplicațiilor enterprise. Aceasta permite gestionarea eficientă a logicii aplicației, comunicarea cu baza de date și implementarea mecanismelor de securitate și autentificare.

Pentru stocarea și administrarea datelor a fost ales sistemul de gestiune a bazelor de date relaționale PostgreSQL, împreună cu extensia PostGIS, utilizată pentru procesarea și manipularea datelor geospațiale. Integrarea acestor tehnologii este esențială în contextul unei aplicații turistice, deoarece permite realizarea unor funcționalități bazate pe localizare, precum calcularea distanțelor, afișarea obiectivelor pe hartă și identificarea locațiilor aflate în apropierea utilizatorului.

În subcapitolele următoare vor fi prezentate în detaliu principalele tehnologii utilizate în cadrul aplicației, modul în care acestea au fost integrate și avantajele pe care le oferă în dezvoltarea unei platforme moderne de turism digital.

2.3.1 Frontend -VueJs

Pentru dezvoltarea frontend-ului aplicației TravelS a fost ales framework-ul Vue.js, mai exact versiunea Vue 3, datorită flexibilității sale, performanței ridicate și posibilității de dezvoltare a unor interfețe moderne și interactive. Frontend-ul reprezintă componenta responsabilă de interacțiunea directă cu utilizatorul, având rolul de a afișa informațiile primite de la server, de a prelua date introduse de utilizator și de a asigura o experiență intuitivă și fluidă în utilizarea aplicației.

În contextul aplicațiilor web moderne, frontend-ul nu mai are doar rolul de afișare statică a informațiilor, ci devine un sistem complex care gestionează starea aplicației, actualizarea dinamică a conținutului și comunicarea permanentă cu serviciile backend. Alegerea unei tehnologii potrivite pentru această componentă este esențială, deoarece influențează atât experiența utilizatorului, cât și procesul de dezvoltare și mentenanță a aplicației.

Vue.js este un framework JavaScript open-source dezvoltat pentru construirea interfețelor utilizator și a aplicațiilor Single Page Application (SPA)[6]. O aplicație de tip Single Page Application încarcă o singură pagină HTML inițială și actualizează conținutul în mod dinamic fără reîncărcarea completă a paginii. Această abordare reduce timpul de răspuns și oferă o experiență mai apropiată de cea a aplicațiilor desktop.

În cadrul aplicației, utilizarea unui model SPA a fost importantă deoarece utilizatorii interacționează frecvent cu diferite componente ale sistemului: navigarea între obiective turistice, aplicarea filtrelor, vizualizarea itinerariilor, accesarea chatbot-ului sau încărcarea

informațiilor despre locații. Reîncărcarea completă a paginii pentru fiecare acțiune ar fi redus semnificativ performanța și experiența utilizatorului.

Unul dintre principalele motive pentru alegerea Vue 3 îl reprezintă introducerea mecanismului Composition API. În versiunile anterioare ale framework-ului, organizarea codului se realiza utilizând Options API, unde logica unei componente era separată în secțiuni precum data, methods, computed sau watch. În aplicațiile complexe, această abordare poate conduce la dispersarea logicii și la dificultăți în reutilizarea codului. [17] Diferența dintre aceste mecanisme se poate observa în cadrul Figurii 4.

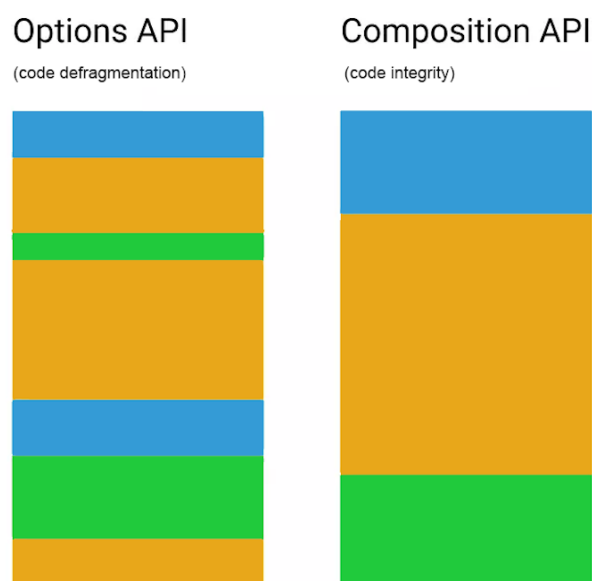


Figura 4 – Diferența dintre Option și Composition API

Composition API propune o modalitate modernă de structurare a codului, prin gruparea elementelor care aparțin aceleiași funcționalități, indiferent dacă acestea reprezintă date reactive, metode sau valori calculate. Astfel, logica unei componente devine mai ușor de urmărit, reutilizat și întreținut, în special în cazul aplicațiilor cu un grad ridicat de complexitate.

Această abordare a fost utilizată pentru gestionarea componentelor responsabile de filtrarea obiectivelor turistice, gestionarea datelor utilizatorilor și comunicarea cu serviciile API. Astfel, logica a fost împărțită în module independente și reutilizabile, reducând redundanța codului și facilitând extinderea ulterioară a aplicației.

Un alt avantaj important oferit de Vue.js este sistemul de reactivitate. Acesta permite actualizarea automată a interfeței în momentul modificării datelor. În locul actualizării manuale

a elementelor din pagină, framework-ul monitorizează modificările și actualizează automat doar componentele afectate.

În aplicația TravelS, mecanismul de reactivitate este utilizat în numeroase scenarii. Spre exemplu, atunci când utilizatorul aplică filtre pentru afișarea obiectivelor turistice, lista este actualizată instantaneu fără reîncărcarea paginii. Similar, la modificarea criteriilor de căutare sau selectarea unei categorii noi, informațiile sunt actualizate automat în interfață.

Comunicarea dintre frontend și backend a fost realizată prin intermediul cererilor HTTP către API-ul dezvoltat în C# ASP.NET. Pentru realizarea acestor operații a fost utilizată biblioteca FetchJS, care facilitează trimiterea cererilor și gestionarea răspunsurilor primite de la server.

Fluxul general de funcționare al comunicării dintre interfața utilizator și backend pornește de la o acțiune inițiată de utilizator în cadrul aplicației. Componenta Vue preia datele introduse sau selecțiile realizate în interfață și transmite o cerere HTTP către API-ul dezvoltat în ASP.NET Core. Serverul primește cererea, procesează informațiile și, atunci când este necesar, interoghează baza de date pentru obținerea sau modificarea datelor. După finalizarea procesării, rezultatul este transmis înapoi către frontend, iar interfața se actualizează automat, fără a fi necesară reîncărcarea completă a paginii. Acest flux de comunicare este reprezentat în Figura 5.

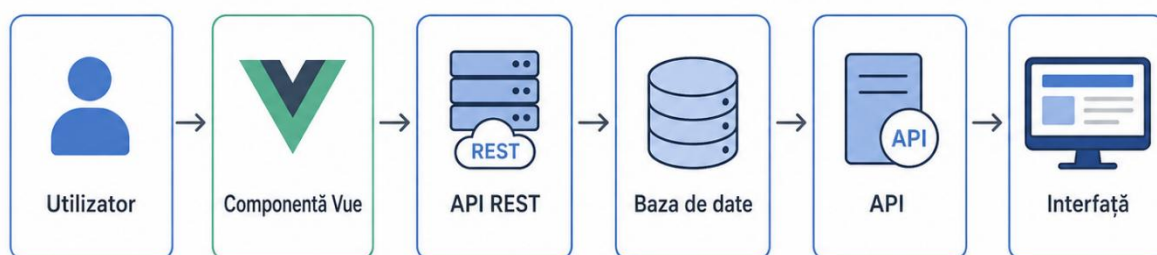


Figura 5 – Comunicarea FE & BE

Pentru gestionarea navigării între diferitele pagini ale aplicației a fost utilizată biblioteca Vue Router[7]. Aceasta permite definirea rutelor și încărcarea componentelor asociate fiecărei secțiuni din aplicație.

În cadrul aplicației TravelS au fost definite rute pentru principalele funcționalități:

- pagina principală;

- pagina de autentificare;
- afișarea obiectivelor turistice;
- administrarea conținutului;
- itinerarii;
- profilul utilizatorului.

Utilizarea Vue Router a permis implementarea mecanismelor de protecție a rutelor prin route guards. Aceste mecanisme verifică dacă utilizatorul este autentificat înainte de accesarea anumitor componente.

De exemplu, accesul la panoul de administrare este permis doar utilizatorilor care dețin rolul de administrator. În momentul accesării unei rute protejate, aplicația verifică existența tokenului JWT și drepturile asociate utilizatorului.

Un alt aspect important al dezvoltării frontend-ului a fost utilizarea bibliotecii PrimeVue pentru construirea interfeței grafice. PrimeVue oferă un număr mare de componente reutilizabile, precum:

- tabele;
- formulare;
- meniuri;
- ferestre de dialog;
- calendare;
- componente de navigare;
- elemente de filtrare.

Utilizarea acestor componente a redus timpul necesar dezvoltării și a asigurat o consistență vizuală la nivelul întregii aplicații.

Pentru organizarea layout-ului și implementarea unui design responsive a fost utilizat PrimeFlex, un sistem bazat pe clase utilitare CSS. Acesta permite adaptarea automată a interfeței pentru diferite dimensiuni ale ecranului, inclusiv dispozitive mobile și tablete.

În dezvoltarea aplicației a fost utilizat și Vite, un instrument modern pentru compilarea și rularea aplicațiilor frontend. Comparativ cu soluțiile tradiționale, Vite oferă timpi de pornire foarte reduși și suport pentru Hot Module Replacement (HMR), mecanism care permite actualizarea instantanee a componentelor fără reîncărcarea întregii aplicații.

Acest lucru a avut un impact direct asupra procesului de dezvoltare, deoarece modificările realizate în cod puteau fi observate imediat, reducând timpul necesar testării și creșterii productivității.

Prin utilizarea Vue.js și a ecosistemului asociat, frontend-ul aplicației TravelS a fost dezvoltat utilizând o arhitectură modulară, performantă și ușor de extins. Alegerea acestor tehnologii a contribuit la construirea unei interfețe interactive și intuitive, capabile să ofere utilizatorilor o experiență fluidă și eficientă în utilizarea platformei turistice.

2.3.2 Backend – C# .NET API

Backend-ul aplicației TravelS reprezintă nucleul logic al sistemului, având rolul de a procesa datele primite din interfața utilizatorului, de a realiza operații asupra bazei de date și de a furniza informațiile necesare către frontend. În cadrul unei aplicații moderne, backend-ul nu se limitează doar la stocarea și returnarea datelor, ci include mecanisme complexe de validare, securitate, autentificare, gestionare a proceselor și integrare cu servicii externe.

Pentru dezvoltarea acestei componente a fost utilizată platforma C# ASP.NET Web API [8], o tehnologie dezvoltată de Microsoft, utilizată pe scară largă pentru construirea serviciilor web și a aplicațiilor distribuite. Alegerea acestei tehnologii a fost determinată de mai mulți factori, printre care performanța ridicată, arhitectura flexibilă, suportul extins pentru dezvoltarea serviciilor REST și integrarea eficientă cu alte tehnologii din ecosistemul .NET.

ASP.NET Web API permite dezvoltarea unor servicii capabile să comunice cu diferite tipuri de clienți, precum aplicații web, aplicații mobile sau alte sisteme software. În cazul aplicației TravelS, API-ul are rolul de intermediar între componenta frontend realizată în Vue.js și baza de date PostgreSQL, asigurând transferul și procesarea informațiilor într-un mod sigur și eficient.

Arhitectura implementată în cadrul aplicației urmează modelul REST (Representational State Transfer), unul dintre cele mai utilizate stiluri arhitecturale pentru dezvoltarea serviciilor

web moderne. Acest model permite accesarea resurselor prin intermediul unor endpoint-uri specifice și utilizează metode HTTP standard.

Principalele metode utilizate în aplicația TravelS sunt:

- **GET** – pentru obținerea informațiilor;
- **POST** – pentru adăugarea unor date noi;
- **PUT** – pentru actualizarea informațiilor existente;
- **DELETE** – pentru ștergerea datelor.

Fluxul general de funcționare al unui API REST în cadrul aplicației este reprezentat în Figura 6.

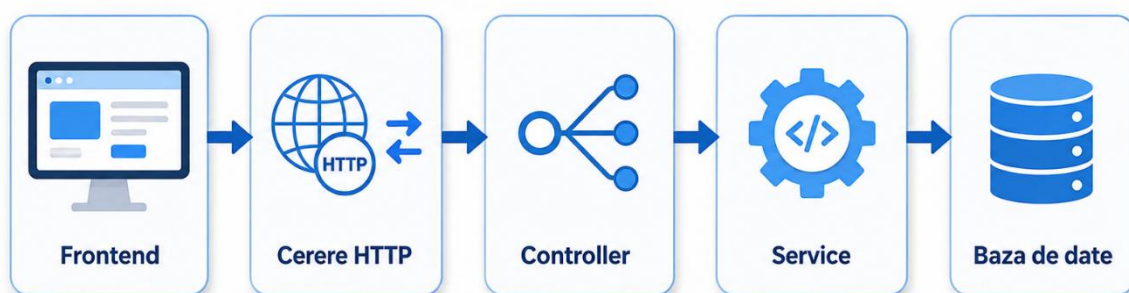


Figura 6 – Fluxul pentru REST API

În acest proces, utilizatorul realizează o acțiune în interfață, iar frontend-ul transmite o cerere către backend. Cererea este preluată de un controller, care transmite mai departe informațiile către serviciul corespunzător. Acesta aplică logica necesară și interacționează cu baza de date prin intermediul service-ului. După procesarea informațiilor, rezultatul este transmis înapoi către utilizator.

Pentru organizarea codului și menținerea unei arhitecturi clare și scalabile, aplicația a fost dezvoltată utilizând o separare pe straturi (Layered Architecture). Această abordare permite împărțirea responsabilităților și reduce dependențele dintre componente.

Structura generală utilizată în aplicație este formată din următoarele componente:

Controllers

Controller-ele reprezintă punctele de intrare ale cererilor HTTP și gestionează comunicarea directă cu clientul. Acestea primesc datele din cerere și apelează serviciile necesare.

```
[Route("api/[controller]")]
[ApiController]
[Authorize]

public class ObjectivesController : ControllerBase
{
    [HttpPost("GetLocalObjectives")]
    public async Task<ServiceResult<List<ObjectiveModel>>> GetLocalObjectives
        (ObjectiveFilterModel filter)
    {
        return await _objectiveService.GetLocalObjectives(filter);
    }
}
```

Services

Serviciile conțin logica de business a aplicației și procesează informațiile necesare înainte de accesarea bazei de date.

Exemple de operații realizate în cadrul serviciilor:

- validarea informațiilor introduse;
- filtrarea obiectivelor turistice;
- generarea recomandărilor;
- verificarea drepturilor utilizatorilor;

Validările backend au rolul de a împiedica introducerea unor date invalide sau incomplete și de a preveni apariția unor erori la nivelul sistemului.

În cazul identificării unei erori, sistemul transmite răspunsuri standardizate utilizând codurile HTTP corespunzătoare.

Exemple:

- **200 OK** – cerere procesată cu succes;
- **201 Created** – resursă creată;
- **400 Bad Request** – date invalide;

- **401 Unauthorized** – utilizator neautentificat;
- **403 Forbidden** – acces interzis;
- **404 Not Found** – resursa nu există;
- **500 Internal Server Error** – eroare internă.

Securitatea reprezintă un aspect esențial în cadrul oricărei aplicații moderne. În cadrul aplicației TravelS a fost implementat un mecanism de autentificare bazat pe token-uri JWT (JSON Web Token).

Procesul de autentificare poate fi descris prin următoarele etape:

1. Utilizatorul introduce datele de autentificare;
2. Cererea este transmisă către backend;
3. Serverul verifică informațiile în baza de date;
4. Dacă autentificarea este validă, se generează un token JWT;
5. Tokenul este transmis către client;
6. Clientul atașează tokenul în cererile viitoare.

Prin utilizarea token-urilor JWT, sistemul poate identifica utilizatorul fără a păstra informațiile de sesiune pe server, reducând consumul de resurse și crescând scalabilitatea aplicației.

Prin utilizarea platformei ASP.NET Web API a fost posibilă dezvoltarea unui backend robust, modular și scalabil, capabil să gestioneze eficient procese complexe și volume mari de informații.

2.3.3 PostgreSQL și PostGIS

Baza de date reprezintă una dintre componentele fundamentale ale unei aplicații informatice, având rolul de a stoca, organiza și gestiona informațiile utilizate de sistem. Aceasta are responsabilitatea de a păstra datele referitoare la utilizatori, obiective turistice, evenimente, itinerarii, coordonate geografice și alte informații necesare funcționării platformei. Alegerea

unei soluții eficiente pentru gestionarea acestor date este esențială, deoarece influențează performanța aplicației, viteza de acces la informații și posibilitatea extinderii ulterioare a sistemului.

Pentru dezvoltarea aplicației a fost ales sistemul de gestiune a bazelor de date PostgreSQL, o soluție open-source recunoscută pentru stabilitatea, performanța și suportul extins pentru operații complexe. PostgreSQL este unul dintre cele mai utilizate sisteme relaționale moderne și este frecvent întâlnit în aplicații care necesită gestionarea unor volume mari de date și un nivel ridicat de flexibilitate.

Comparativ cu alte sisteme de gestiune a bazelor de date relaționale, PostgreSQL oferă multiple avantaje, printre care:

- suport pentru interogări complexe;
- extensibilitate ridicată;
- integritate a datelor;
- mecanisme avansate de indexare;
- suport pentru tipuri de date personalizate;
- compatibilitate cu extensii suplimentare.

Utilizarea PostgreSQL a fost justificată în special de necesitatea gestionării informațiilor geografice și a datelor spațiale asociate obiectivelor turistice.

Structura bazei de date a fost proiectată utilizând modelul relațional, în care informațiile sunt organizate sub formă de tabele interconectate prin relații. Această abordare permite reducerea redundanței datelor și menținerea consistenței informațiilor stocate.

Unele dintre entitățile folosite în cadrul aplicației sunt utilizatorii, obiectivele turistice, categoriile, itinerariile, evenimentele, imaginile și recenziile. Acestea au fost definite în funcție de informațiile necesare pentru funcționarea platformei și pentru organizarea datelor într-un mod clar și eficient. Între aceste entități au fost stabilite relații de tip one to one (1:1), one to many (1:N) și many to many (N:M), în funcție de modul în care datele sunt conectate între ele.

De exemplu, un utilizator poate avea mai multe itinerarii, iar un itinerariu poate conține mai multe obiective turistice.

Pentru interacțiunea cu baza de date a fost utilizat Entity Framework Core, care permite maparea entităților definite în cod către tabelele existente în PostgreSQL.

Această abordare oferă mai multe avantaje în procesul de dezvoltare a aplicației. Prin utilizarea Entity Framework Core se reduce nevoia de a scrie manual cod SQL, deoarece multe operații asupra bazei de date pot fi realizate direct din cod. De asemenea, pot fi utilizate interogări LINQ, ceea ce face accesarea și filtrarea datelor mai ușor de implementat. Un alt avantaj este gestionarea automată a relațiilor dintre entități, precum și suportul pentru migrări, care permite modificarea structurii bazei de date într-un mod controlat. Toate aceste aspecte contribuie la o dezvoltare mai rapidă și mai eficientă a aplicației.

Migrările reprezintă un mecanism important oferit de Entity Framework Core[18] și permit modificarea incrementală a structurii bazei de date fără pierderea datelor existente.

Procesul general de funcționare este următorul:

1. Se definește o entitate în cod;
2. Entity Framework generează structura tablei;
3. Se creează o migrare;
4. Migrarea este aplicată în PostgreSQL;
5. Structura bazei de date este actualizată.

Pe lângă funcționalitățile standard oferite de PostgreSQL, aplicația utilizează extensia PostGIS pentru gestionarea informațiilor geospațiale.

PostGIS reprezintă o extensie a PostgreSQL care adaugă suport pentru stocarea și procesarea datelor geografice și spațiale. Aceasta transformă PostgreSQL într-un sistem capabil să realizeze operații GIS (Geographic Information System), oferind funcționalități avansate pentru manipularea coordonatelor și efectuarea calculelor spațiale.

Integrarea PostGIS a fost necesară deoarece aplicația TravelS gestionează informații despre obiective turistice care conțin coordonate geografice reale. Fiecare obiectiv turistic este asociat cu o poziție geografică reprezentată prin latitudine și longitudine. Aceste informații sunt stocate utilizând tipuri de date spațiale specifice PostGIS cum ar fi proprietatea Point din Figura 7.

```
1 reference  
public Point? Location { get; set; }
```

Figura 7 – Proprietatea Point

Utilizarea tipurilor de date spațiale permite realizarea unor operații care ar fi dificil de implementat prin metode tradiționale.

Printre funcționalitățile implementate cu ajutorul PostGIS se numără:

1. Filtrarea obiectivelor turistice în funcție de proximitate

Aplicația poate identifica obiective aflate într-o anumită rază în jurul utilizatorului.

Exemplu:

- afișarea tuturor obiectivelor aflate la mai puțin de 10 km de locația curentă.

```
if (filter.MaxDistance.HasValue)  
{  
    query = query.Where(o => o.Location.Distance(userLocation) <=  
filter.MaxDistance.Value * 1000); //convertit în km  
}
```

2. Recomandări bazate pe locație

Prin utilizarea informațiilor geografice, aplicația poate genera sugestii adaptate poziției utilizatorului. De exemplu, sistemul poate recomanda obiective turistice apropiate sau poate sugera locații similare din aceeași zonă.

Un alt avantaj important al extensiei PostGIS îl reprezintă utilizarea indexării spațiale. În bazele de date tradiționale, căutarea informațiilor geografice în volume mari de date poate deveni costisitoare din punct de vedere computațional. Pentru optimizarea performanței, PostGIS utilizează indexuri de tip GiST (Generalized Search Tree), care reduc semnificativ timpul necesar executării interogărilor spațiale.

Prin utilizarea PostgreSQL împreună cu extensia PostGIS, aplicația beneficiază de o infrastructură robustă pentru gestionarea informațiilor geografice și a relațiilor dintre date. Această combinație permite dezvoltarea unor funcționalități avansate bazate pe localizare și oferă suport pentru extinderea ulterioară a platformei, menținând în același timp performanța și integritatea datelor.

2.3.4 LM Studio și modele LLM

Un element important al aplicației TravelS îl reprezintă integrarea unei componente bazate pe inteligență artificială, utilizată pentru dezvoltarea unui chatbot capabil să ofere informații relevante utilizatorilor într-un mod conversațional. Scopul acestei componente este de a îmbunătăți experiența utilizatorului prin furnizarea rapidă a răspunsurilor și prin reducerea timpului necesar căutării manuale a informațiilor despre obiective turistice, locații sau alte elemente disponibile în platformă.

În ultimii ani, dezvoltarea modelelor lingvistice de mari dimensiuni (Large Language Models – LLM) a schimbat semnificativ modul în care utilizatorii interacționează cu sistemele informatice. Aceste modele utilizează tehnici avansate de învățare profundă pentru procesarea și generarea limbajului natural, având capacitatea de a înțelege contextul unei conversații și de a genera răspunsuri asemănătoare celor formulate de un utilizator uman [9].

Modelele LLM sunt antrenate pe volume foarte mari de date textuale și învață relațiile dintre cuvinte, propoziții și contexte. Spre deosebire de sistemele tradiționale bazate pe reguli fixe sau pe căutări după cuvinte cheie, aceste modele utilizează relații semantice complexe pentru interpretarea informațiilor.

Utilizarea modelelor LLM a fost motivată de necesitatea dezvoltării unei interacțiuni mai naturale și mai intuitive între utilizator și sistem. În locul unei căutări clasice, utilizatorul poate adresa întrebări folosind limbaj natural, iar sistemul încearcă să identifice informațiile relevante și să formuleze un răspuns adecvat.

Pentru implementarea acestei componente a fost utilizată aplicația LM Studio. LM Studio reprezintă o platformă software care permite rularea locală a modelelor lingvistice fără necesitatea utilizării unor servicii externe sau a unor API-uri comerciale.

Utilizarea LM Studio a adus mai multe avantaje în cadrul proiectului. Un prim avantaj este posibilitatea rulării locale a modelelor lingvistice, fără dependența directă de servicii externe sau de API-uri comerciale. Acest lucru contribuie la eliminarea costurilor asociate utilizării unor servicii cloud și oferă un control mai mare asupra modului în care sunt procesate datele. De asemenea, rularea locală ajută la protejarea informațiilor utilizatorilor, deoarece întrebările și datele transmise către chatbot nu trebuie trimise către servere externe. Un alt beneficiu important este posibilitatea schimbării rapide a modelelor utilizate, în funcție de performanță

sau de cerințele aplicației. În plus, integrarea locală poate contribui la reducerea timpului de răspuns, deoarece comunicarea se realizează în cadrul aceluiași mediu de lucru.

Un alt avantaj important al rulării locale îl reprezintă controlul complet asupra datelor procesate. În cazul serviciilor cloud, întrebările utilizatorilor și informațiile procesate sunt transmise către servere externe. Prin utilizarea LM Studio, procesarea poate fi realizată local, reducând problemele legate de confidențialitatea informațiilor și dependența de conexiuni externe. LM Studio funcționează ca un serviciu separat care expune un API local și comunică cu backend-ul aplicației. Fluxul de date este reprezentat mai jos.

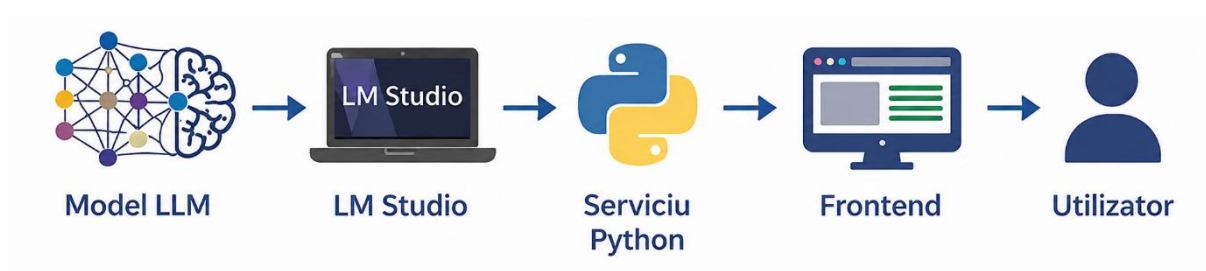


Figura 8 – Fluxul de integrare al chatbot-ului

În cadrul aplicației nu a fost utilizată abordarea clasică în care modelul generează răspunsuri exclusiv pe baza informațiilor învățate în timpul antrenării. O astfel de metodă poate conduce la apariția fenomenului cunoscut sub denumirea de *hallucination*, situație în care modelul produce informații incorecte sau inexistente.

Pentru reducerea acestui risc a fost utilizată o abordare bazată pe *Retrieval-Augmented Generation* (RAG). Această metodă combină modelele lingvistice cu mecanisme de căutare semantică, permițând modelului să utilizeze informații reale extrase dintr-o bază de cunoștințe.

În această abordare, informațiile utilizate de chatbot nu provin exclusiv din cunoștințele modelului, ci și din documentele introduse în sistem.

Procesul de funcționare începe prin încărcarea documentelor turistice în sistem, acestea reprezentând sursa principală de informații utilizată de chatbot. După încărcare, textele sunt împărțite în segmente mai mici, pentru a putea fi procesate mai ușor și pentru a permite identificarea mai precisă a informațiilor relevante. Fiecare segment este apoi convertit într-un vector numeric, numit *embedding*, care reprezintă sensul textului într-o formă matematică. Acești vectori sunt stocați într-o bază vectorială, unde pot fi comparați ulterior cu întrebările

utilizatorilor. Atunci când utilizatorul introduce o întrebare, aceasta este transformată la rândul ei într-un embedding, iar sistemul caută vectorii cei mai similari din baza de date. Documentele relevante identificate sunt transmise către modelul lingvistic, care le folosește ca sursă de context pentru generarea răspunsului final.

Embedding-urile reprezintă valori numerice ale informațiilor textuale. Acestea transformă cuvintele și propozițiile în vectori matematici multidimensionali, astfel încât texte cu sens apropiat să fie poziționate mai aproape în spațiul vectorial. (Vezi exemplu în Figura 9)

Această metodă permite identificarea informațiilor relevante chiar și atunci când utilizatorii formulează întrebările în mod diferit.

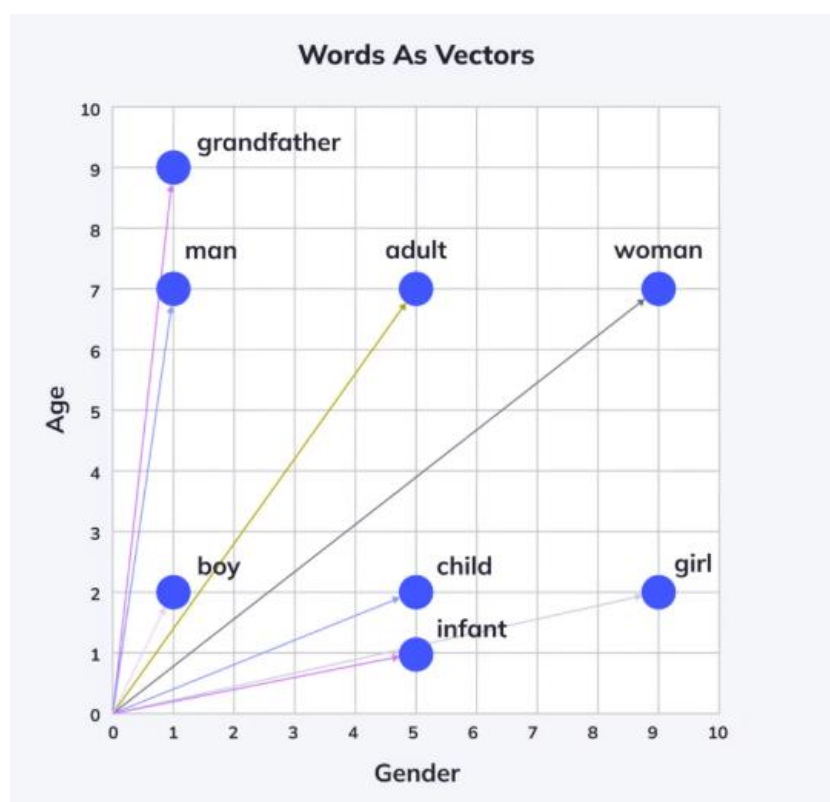


Figura 9 – Exemplu grafic pentru Embedding

De exemplu:

Întrebare 1: „Ce pot vizita în Sibiu?”

Întrebare 2: „Care sunt cele mai interesante locuri turistice din Sibiu?”

Deși formulările sunt diferite, reprezentările vectoriale generate pot avea valori apropiate, iar sistemul poate identifica aceeași intenție și aceleași informații relevante.

Separarea componentei AI într-un serviciu independent reprezintă o decizie arhitecturală importantă deoarece permite modificarea ulterioară a modelelor utilizate fără a afecta celelalte componente ale aplicației. Astfel, pot fi integrate modele noi sau pot fi schimbate mecanismele de procesare fără modificări majore asupra backend-ului principal.

Prin integrarea LM Studio și a modelelor LLM, aplicația TravelS oferă o modalitate modernă și flexibilă de interacțiune cu utilizatorii, contribuind la creșterea gradului de accesibilitate și la furnizarea unor răspunsuri mai naturale și mai relevante în procesul de explorare a informațiilor turistice.

3. Rezolvarea temei de proiect

3.1 Definirea cerințelor prin user stories și planificarea funcționalităților

Primul pas esențial în procesul de dezvoltare a aplicației TravelS a fost identificarea și definirea cerințelor funcționale din perspectiva utilizatorului final. Pentru realizarea unei aplicații coerente, structurate și orientate către nevoile reale ale utilizatorilor, a fost utilizată metoda user stories, o abordare specifică metodologiilor Agile și dezvoltării software centrate pe utilizator. Această metodă permite descrierea funcționalităților într-o formă simplă și clară, din perspectiva utilizatorului care interacționează direct cu sistemul.

Importanța utilizării user stories este evidențiată și în articolul [5], unde autorii subliniază faptul că acestea contribuie la definirea mai clară a cerințelor software, facilitează comunicarea dintre utilizatori și echipa de dezvoltare și susțin procesul de dezvoltare incrementală al aplicațiilor. Prin utilizarea user stories, funcționalitățile aplicației pot fi organizate și prioritizate mai eficient, reducând riscul interpretării greșite a cerințelor și contribuind la dezvoltarea unor soluții software mai apropiate de nevoile reale ale utilizatorilor.



Figura 10 – Exemplu de User Story

User stories reprezintă enunțuri scurte, standardizate, prin care sunt descrise așteptările utilizatorului față de sistem, în contextul unei acțiuni concrete și a unui beneficiu specific. Ele sunt formulate sub forma: „Ca și utilizator, vreau să... astfel încât să...” (Figura 10), ceea ce oferă atât perspectiva actorului implicat, cât și scopul clar al acțiunii

Prin acest demers, am conturat o serie de interacțiuni fundamentale între utilizator și platformă, care au acoperit atât nevoile de bază precum căutarea, filtrarea și afișarea obiectivelor turistice cât și funcționalități mai avansate, printre care se numără generarea de itinerarii, vizualizarea rutelor și integrarea unui sistem de întrebări și răspunsuri. În plus, am avut în vedere extinderea aplicației și către o dimensiune colaborativă, unde utilizatorii pot încărca propriile experiențe multimedia și pot contribui activ la îmbunătățirea conținutului.

Prin user stories am stabilit așteptările utilizatorului obișnuit, turistul, dar și rolurile secundare, cum ar fi administratorul, care are acces la funcționalități specifice de gestionare a conținutului. Aceste roluri au fost indispensabile pentru modelarea logicii aplicației, definirea bazelor de date și dezvoltarea interfeței grafice adaptate fiecărui tip de utilizator. Tabelul complet cu toate user stories definite pentru aplicația TravelS este prezent în Anexa 1.

Pe baza acestor user stories, am structurat un set de funcționalități planificate, denumite planned features, care au fost grupate în funcție de complexitatea estimată a implementării: ușoră, medie sau dificilă. Această etapă de planificare a avut scopul de a estima efortul de

dezvoltare pentru fiecare componentă în parte și de a organiza un roadmap realist, respectând principiile dezvoltării incrementale.

Printre primele funcționalități esențiale s-a numărat implementarea unui sistem de căutare și filtrare a obiectivelor turistice, ce oferă utilizatorului posibilitatea de a restrânge lista locurilor vizibile în funcție de interesele personale, cum ar fi: muzee, parcuri, monumente sau alte tipuri de obiective. Această funcționalitate a fost considerată de dificultate redusă, dar extrem de importantă pentru ușurința în utilizare și pentru experiența generală a platformei.

Un alt set de funcționalități a vizat vizualizarea locațiilor turistice pe hartă și integrarea cu Google Maps, oferind utilizatorului posibilitatea de a vedea rutele către un anumit obiectiv, pornind de la locația sa curentă. În plus, s-a dorit afișarea estimărilor de timp pentru deplasare, în funcție de mijlocul de transport (mers pe jos, cu mașina, transport public), pentru a sprijini planificarea realistă a călătoriilor. Aceste componente au fost clasificate ca având o complexitate medie, necesitând integrarea cu servicii externe și un management riguros al datelor geospațiale.

Pentru o experiență turistică personalizată, utilizatorul are posibilitatea de a crea și gestiona propriul itinerariu, alegând din lista de obiective cele care îl interesează și adăugându-le într-un plan de vizită. Această funcționalitate nu doar că oferă un mod eficient de organizare, dar permite și modificarea dinamică a planului, în funcție de timpul disponibil sau de schimbările apărute în programul utilizatorului.

De asemenea, aplicația include o secțiune dedicată evenimentelor locale, care oferă informații despre activități culturale, festivaluri sau târguri care au loc într-un anumit oraș. Pentru fiecare eveniment, utilizatorul poate consulta detalii suplimentare precum locația exactă, data desfășurării, intervalul orar, costul biletelor sau modalitățile de participare.

Un aspect important al platformei îl constituie componenta interactivă și colaborativă. Astfel, utilizatorii pot pune întrebări despre un obiectiv turistic, pot răspunde la întrebările altora sau pot consulta o secțiune de întrebări frecvente, cu informații utile despre obiective populare. Acest sistem încurajează crearea unei comunități active în jurul aplicației, în care experiențele personale contribuie la îmbunătățirea informațiilor disponibile. În plus, aplicația oferă o galerie

multimedia pentru fiecare obiectiv turistic, permițând afișarea de imagini și videoclipuri relevante. Utilizatorii pot, de asemenea, încărca propriile fișiere media din călătoriile lor, contribuind la o prezentare mai realistă și mai autentică a locațiilor vizitate. Această funcționalitate a fost clasificată ca fiind una dintre cele mai dificile din punct de vedere tehnic, implicând aspecte legate de stocare, optimizare, moderare și afișare.

Pentru o experiență personalizată, aplicația permite marcarea obiectivelor favorite, oferind utilizatorului posibilitatea de a salva într-un loc separat acele locații pe care intenționează să le viziteze sau care l-au impresionat.

Nu în ultimul rând, aplicația oferă un panou de administrare, prin care un utilizator cu rol de administrator poate gestiona toate componentele esențiale ale platformei: obiectivele turistice, imaginile atașate, recenziile, evenimentele locale, itinerariile și conturile utilizatorilor. De asemenea, utilizatorii pot sugera noi obiective sau evenimente, care ulterior sunt revizuite și aprobate de un administrator, contribuind astfel la menținerea actualizată a conținutului și la implicarea directă a comunității în evoluția aplicației.

Această etapă de definire a funcționalităților, construită în jurul user stories-urilor și organizată după gradul estimat de complexitate al fiecărei componente, a reprezentat fundamentul logic pe care s-a construit întreaga aplicație. Formularea cerințelor din perspectiva utilizatorului a permis nu doar conturarea clară a nevoilor funcționale, ci și o mai bună înțelegere a modului în care aceste nevoi pot fi transformate în soluții tehnice viabile. Pe baza acestei analize, a fost posibilă o planificare realistă a etapelor de dezvoltare, adaptată la resursele disponibile și la timpul limitat avut la dispoziție pentru realizarea proiectului.

Într-un context în care dezvoltarea s-a desfășurat într-un interval de timp bine determinat, a fost esențial să stabilesc o serie de priorități. Astfel, funcționalitățile de bază, precum căutarea și filtrarea obiectivelor turistice, afișarea lor pe hartă, crearea de itinerarii și vizualizarea evenimentelor locale, au fost implementate cu prioritate, deoarece ele asigură nucleul experienței utilizatorului și reflectă direct scopul principal al aplicației. Alte funcționalități, considerate de dificultate mai ridicată sau cu impact mai redus asupra fluxului principal de utilizare cum ar fi sistemul colaborativ de întrebări și răspunsuri au fost planificate pentru etape ulterioare, într-o logică de extindere modulară a platformei.

Această abordare incrementală, bazată pe definirea clară a nevoilor și pe selecția funcționalităților în funcție de importanță și efort estimat, a permis menținerea unui echilibru între complexitatea proiectului și fezabilitatea sa practică. În același timp, a asigurat o direcție constantă de dezvoltare orientată spre utilizator, care rămâne obiectivul central în realizarea oricărei aplicații destinate publicului larg.

3.2 Modelarea bazei de date

În cadrul aplicației TravelS, componenta responsabilă de stocarea și administrarea datelor a fost implementată utilizând sistemul de gestiune a bazelor de date PostgreSQL, împreună cu extensia PostGIS. Alegerea acestei soluții a fost determinată de necesitatea utilizării unei baze de date moderne, stabile și performante, capabile să gestioneze atât informații relaționale clasice, cât și date geografice necesare unei aplicații turistice orientate pe locație.

Pentru o platformă precum TravelS, în care utilizatorii pot consulta obiective turistice, evenimente și experiențe aflate în diverse zone geografice, baza de date joacă un rol esențial. Aceasta trebuie să permită nu doar salvarea rapidă a informațiilor, ci și interogări eficiente, filtrări complexe și calcule spațiale. Din acest motiv, combinația PostgreSQL și PostGIS s-a dovedit o alegere potrivită.

3.2.1 Crearea bazei de date inițiale

Primul pas în implementarea componentei de stocare a fost crearea unei baze de date inițial goale în mediul PostgreSQL. Această bază de date a reprezentat spațiul principal în care urmau să fie generate tabelele aplicației prin intermediul Entity Framework Core.

Baza de date a fost creată utilizând pgAdmin, interfață grafică oficială pentru administrarea serverelor PostgreSQL. Prin intermediul acestei aplicații au fost realizate operațiile inițiale de creare a bazei de date, configurare a utilizatorilor și monitorizare a obiectelor generate ulterior de aplicație.

După crearea bazei de date, aceasta nu conținea tabele sau relații, urmând ca structura completă să fie construită automat prin migrări Entity Framework Core.

3.2.2 Conectarea backend-ului la PostgreSQL

Pentru ca aplicația ASP.NET Core să poată comunica cu serverul PostgreSQL, a fost configurat un connection string în fișierul appsettings.json.

```
"ConnectionStrings": {  
    "DefaultConnection": "Host=localhost;Port=5432;Database=TravelS4;Username=postgres;Password=1234"  
},
```

Acest connection string conține informațiile esențiale necesare conectării:

- **Host** – serverul pe care rulează baza de date
- **Port** – portul standard PostgreSQL, 5432
- **Database** – numele bazei de date utilizate
- **Username** – utilizatorul de conectare
- **Password** – parola contului de acces

În mediul local de dezvoltare aceste informații au fost utilizate direct pentru testare și implementare rapidă.

Totuși, într-un mediu real de producție, afișarea explicită a datelor sensibile în fișiere de configurare nu este recomandată. Din acest motiv, ulterior connection string-ul poate fi securizat prin mecanisme de criptare sau stocare în variabile de mediu (.env).

Această abordare previne expunerea accidentală a datelor de conectare și crește nivelul general de securitate al aplicației.

3.2.3 Comunicarea dintre C# și PostgreSQL

Legătura dintre backend-ul implementat în C# și serverul PostgreSQL a fost realizată prin intermediul mai multor librării specializate, integrate în ecosistemul .NET.

Printre cele mai importante se numără:

- **EntityFrameworkCore** – ORM principal utilizat pentru maparea claselor C# către tabele relaționale

- **Microsoft.EntityFrameworkCore** – nucleul framework-ului ORM
- **Npgsql.EntityFrameworkCore.PostgreSQL** – provider oficial care permite conectarea Entity Framework Core la PostgreSQL
- **Npgsql** – driver ADO.NET pentru comunicarea directă cu PostgreSQL
- **NetTopologySuite** – bibliotecă utilizată pentru tipuri geometrice și operații spațiale
- **Npgsql.EntityFrameworkCore.PostgreSQL.NetTopologySuite** – extensie care permite integrarea PostGIS în Entity Framework Core

Prin utilizarea acestor pachete, backend-ul poate comunica cu PostgreSQL în mod natural, utilizând obiecte C#, expresii LINQ și migrări automate [10].

3.2.4 Rolul Entity Framework Core în gestionarea datelor

Așa cum a fost menționat anterior, toate operațiile asupra bazei de date au fost administrate prin Entity Framework Core. Acesta elimină necesitatea scrierii manuale a majorității interogărilor SQL și permite manipularea datelor într-un mod orientat pe obiecte.

Astfel, orice dezvoltator poate lucra cu obiecte C# și colecții, iar framework-ul generează automat comenzile SQL necesare în fundal.

De exemplu, următoarea interogare LINQ:

```
var query = _context.Objectives
    .AsNoTracking()
    .Where(x=>x.City=="Sibiu")
    .OrderBy(x=>x.Name)
    .ToList();
```

este transformată automat într-o interogare SQL similară:

```
SELECT * FROM "Objectives" WHERE "City" = 'Sibiu' ORDER BY "Name";
```

Acest mecanism reprezintă unul dintre cele mai importante avantaje ale utilizării Entity Framework Core, deoarece contribuie la reducerea timpului de dezvoltare și simplifică interacțiunea dintre aplicație și baza de date. Prin utilizarea acestuia, dezvoltatorul nu mai este nevoit să scrie manual un volum mare de cod SQL, ceea ce reduce riscul apariției erorilor de sintaxă. În același timp, Entity Framework Core permite modificarea și refactorizarea mai

ușoară a codului, deoarece logica aplicației este integrată într-un mod natural cu structura bazei de date.

În plus, pentru interogări complexe, Entity Framework poate genera join-uri, subinterogări și agregări fără intervenție manuală.

```
var query = _context.Objectives
    .AsNoTracking()
    .Include(x => x.ObjectiveType)
    .Include(x => x.Images)
    .AsQueryable();
```

3.2.5 Sistemul de migrări

După conectarea aplicației la PostgreSQL, structura bazei de date a fost generată automat prin sistemul de migrări oferit de Entity Framework Core.

Migrările reprezintă fișiere care descriu evoluția bazei de date în timp. Ori de câte ori se adaugă o entitate nouă, o coloană suplimentară sau o relație între tabele, Entity Framework poate genera scripturile necesare actualizării structurii.

Exemple de comenzi utilizate:

- Add-Migration – adaugă migrarea în funcție de entitățile aflate în context
- Update-Database – aplică migrările pe baza de date
- Remove-Migration – șterge ultima migrare

Aceste metode oferă control total asupra structurii bazei de date și păstrează istoricul tuturor modificărilor realizate în proiect.

Exemplu conținut migrare:

```
migrationBuilder.CreateTable(
    name: "Experiences",
    columns: table => new
    {
        Id = table.Column<int>(type: "integer", nullable: false)
            .Annotation("Npgsql:ValueGenerationStrategy",
NpgsqlValueGenerationStrategy.IdentityByDefaultColumn),
        Name = table.Column<string>(type: "character varying(100)", maxLength: 100,
nullable: true),
        Title = table.Column<string>(type: "character varying(100)", maxLength: 100,
nullable: false),
        Description = table.Column<string>(type: "text", nullable: false),
        Latitude = table.Column<double>(type: "double precision", nullable: false),
        Longitude = table.Column<double>(type: "double precision", nullable: false),
        City = table.Column<string>(type: "character varying(100)", maxLength: 100,
nullable: true),
        Country = table.Column<string>(type: "character varying(100)", maxLength: 100,
nullable: true),
```

```

        LocationName = table.Column<string>(type: "text", nullable: true),
        Rating = table.Column<int>(type: "integer", nullable: true),
        IsPublic = table.Column<bool>(type: "boolean", nullable: false),
        CreatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable:
false),
        UpdatedAt = table.Column<DateTime>(type: "timestamp with time zone", nullable:
false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Experiences", x => x.Id);
    });

```

3.2.6 Instalarea extensiei PostGIS

Pentru o aplicație turistică modernă, simpla stocare a latitudinii și longitudinii nu este suficientă. Este necesară posibilitatea efectuării de calcule geografice reale: distanțe între puncte, căutări în proximitate, filtrări după rază și interogări spațiale avansate.

Din acest motiv a fost instalată extensia **PostGIS**, una dintre cele mai cunoscute extensii geospațiale pentru PostgreSQL.

PostGIS transformă PostgreSQL într-o bază de date capabilă să gestioneze informații geografice și spațiale, oferind funcționalități specifice sistemelor GIS. Prin utilizarea acestei extensii, aplicația poate lucra cu puncte geografice, linii, trasee și poligoane, elemente importante pentru reprezentarea locațiilor și a zonelor de interes. De asemenea, PostGIS permite realizarea calculelor de distanță, identificarea intersecțiilor spațiale și efectuarea căutărilor în proximitate, cum ar fi afișarea obiectivelor turistice aflate în apropierea utilizatorului. Un alt avantaj important este suportul pentru indexare geografică performantă, care contribuie la executarea mai rapidă a interogărilor spațiale.

Activarea extensiei se realizează prin comanda:

```
CREATE EXTENSION postgis;
```

După activare, baza de date poate stoca tipuri geometrice și poate executa funcții spațiale native.

3.2.7 Utilizarea proprietății Point în entități

Cu ajutorul PostGIS și NetTopologySuite, în entitățile aplicației a fost introdusă proprietatea:

```
public Point? Location { get; set; }
```

Această proprietate a fost utilizată în entități precum Obiective, Evenimente, Experiențe și User.

Tipul Point reprezintă o poziție geografică exactă formată din coordonate longitudine și latitudine. Spre deosebire de simpla stocare separată a valorilor Latitudine și Longitudine, tipul Point permite utilizarea funcțiilor spațiale native oferite de PostGIS precum:

- **ST_Distance(geomA, geomB)** — distanța dintre două puncte.
- **ST_DWithin(geomA, geomB, radius)** — verifică dacă două puncte sunt la o distanță $\leq$ radius.
- **ST_MaxDistance(geomA, geomB)** — distanța maximă (utilă pentru seturi de puncte).

3.2.8 Calcularea distanțelor dintre utilizator și entități

Una dintre cele mai utile funcționalități implementate prin această abordare este calcularea distanței dintre utilizator și diferite obiective turistice, evenimente sau experiențe.

Exemplu conceptual:

```
var nearby = _context.Objectives
    .Where(x=>x.Location.Distance(user.Location) < 5000)
    .ToList();
```

Această interogare poate returna obiective aflate la mai puțin de 5 km față de utilizator.

În fundal, Entity Framework transformă expresia într-o funcție PostGIS de tip ST_Distance sau ST_DWithin.

Avantajele acestei soluții sunt considerabile, deoarece permit dezvoltarea unor funcționalități utile pentru o aplicație turistică modernă. Prin utilizarea datelor spațiale, aplicația poate genera recomandări bazate pe proximitate și poate afișa obiectivele turistice aflate în apropierea utilizatorului. De asemenea, filtrarea după distanță devine mai inteligentă și mai precisă, deoarece sistemul poate calcula poziția obiectivelor în raport cu locația curentă. Un alt avantaj important este performanța ridicată obținută prin indexarea spațială, care permite executarea mai rapidă a interogărilor geografice. Toate aceste aspecte contribuie la oferirea unei experiențe mai personalizate și mai relevante pentru utilizator.

3.3 Implementarea backend-ului în ASP.NET Core

După inițializarea proiectului utilizând platforma ASP.NET Core Web API, unul dintre primii pași realizați în procesul de dezvoltare a aplicației a fost organizarea structurii interne a codului sursă. În cadrul aplicațiilor software moderne, modul în care este structurat proiectul influențează direct ușurința dezvoltării, mentenanței și extinderii ulterioare. O aplicație dezvoltată fără o structură clară poate deveni dificil de administrat pe măsură ce crește în dimensiune și complexitate. Din acest motiv, proiectul a fost organizat încă din fazele inițiale pe directoare distincte, fiecare având responsabilități bine definite.

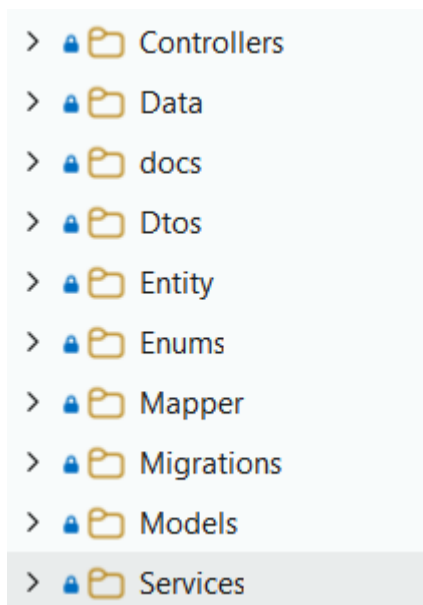


Figura 11 – Structura directoarelor în BE

Codul a fost împărțit în directoare dedicate pentru entități, modele, servicii și controllere (Figura 11). Această separare respectă principiile arhitecturii software moderne și contribuie la claritatea întregului sistem.

Directorul entităților conține clasele care definesc structura datelor persistente salvate în baza de date. Aceste clase sunt asociate direct tabelor relaționale prin intermediul Entity Framework Core și reprezintă modelele reale utilizate de sistem pentru stocarea informațiilor. Fiecare proprietate dintr-o entitate corespunde, în mod normal, unei coloane din baza de date. Prin această abordare, dezvoltatorul poate lucra orientat pe obiecte, fără a fi necesară scrierea manuală a interogărilor SQL pentru operațiile de bază.

Directorul modelelor este utilizat pentru clase auxiliare prin care se transferă date între client și server. Aceste clase sunt utilizate frecvent în operații precum autentificare, înregistrare

sau actualizare profil. Separarea modelelor de entitățile bazei de date oferă un nivel suplimentar de securitate și flexibilitate, deoarece clientul primește doar informațiile necesare, fără acces la structura completă internă a aplicației.

Directorul serviciilor conține logica principală a aplicației. În această zonă sunt implementate operațiile reale ale sistemului: verificări, validări, prelucrări de date, autentificare, comunicare cu baza de date și integrarea cu alte servicii. Prin mutarea logicii în servicii, controller-ele rămân curate și ușor de urmărit, având rolul de a primi cereri și de a transmite răspunsuri.

Directorul controllerelor reprezintă componenta prin care aplicația expune funcționalitățile către exterior. Aici sunt definite endpoint-urile HTTP prin care frontend-ul comunică cu backend-ul. Fiecare controller răspunde pentru o anumită zonă funcțională, cum ar fi utilizatori, autentificare, obiective turistice sau alte resurse.

Această organizare modulară a permis dezvoltarea eficientă a aplicației și a oferit o structură apropiată de standardele utilizate în mediul profesional.

3.3.1 Definirea entității User

Una dintre primele entități implementate în cadrul aplicației a fost entitatea User, utilizată pentru gestionarea conturilor utilizatorilor și a datelor asociate acestora. Existența acestei entități este esențială pentru funcționalități precum autentificare, salvarea preferințelor, administrarea sesiunilor și delimitarea accesului la anumite resurse.

```
namespace TravelSBE.Entity
{
    public class User : BaseAuditEntity
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }
        [Required]
        public string UserName { get; set; }
        public Point? Location { get; set; }
        public string? Email { get; set; }
        public string? Phone { get; set; }
    }
}
```

```

public UserRoleEnum Role { get; set; }
public string Hash { get; set; }
public string Salt { get; set; }
}

```

Clasa User moștenește clasa BaseAuditEntity, ceea ce înseamnă că beneficiază de proprietăți suplimentare utilizate pentru auditarea informațiilor, precum data creării sau data ultimei modificări. Această practică este frecvent utilizată în aplicațiile enterprise, deoarece permite urmărirea istoricului datelor și oferă un control mai bun asupra modificărilor efectuate în sistem.

```

namespace TravelSBE.Entity.Helper
{
    public class BaseAuditEntity
    {
        public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
        public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    }
}

```

Proprietatea Id reprezintă cheia primară a entității. Aceasta identifică în mod unic fiecare utilizator din baza de date și este utilizată în relațiile cu alte tabele. Prin utilizarea atributului Key, Entity Framework înțelege că această proprietate este cheia principală a tabelului. În plus, atributul DatabaseGenerated(DatabaseGeneratedOption.Identity) specifică faptul că valoarea identificatorului este generată automat de baza de date la inserarea unei noi înregistrări. Astfel, fiecare utilizator nou primește automat un identificator numeric unic, fără intervenție manuală.

Proprietatea UserName stochează numele utilizatorului și este marcată ca obligatorie prin atributul Required. Acest lucru înseamnă că sistemul nu permite crearea unui cont fără completarea acestui câmp.

Proprietatea Location utilizează tipul Point și permite salvarea poziției geografice a utilizatorului. Această funcționalitate este utilă în contextul unei aplicații turistice, unde pot fi implementate recomandări bazate pe proximitate sau afișarea punctelor de interes din apropiere.

Proprietățile Email și Phone sunt utilizate pentru informații suplimentare de contact, iar Role definește nivelul de acces al utilizatorului în sistem. Prin intermediul unui enum dedicat, aplicația poate diferenția utilizatorii obișnuiți de administratori sau alte roluri speciale.

Proprietățile Hash și Salt sunt utilizate pentru securizarea parolelor și reprezintă un element critic al sistemului de autentificare.

3.3.2 Implementarea procesului de înregistrare

În cadrul aplicației a fost implementată o metodă de tip SignUp, responsabilă de crearea unui cont nou. Aceasta primește datele completate de utilizator și realizează mai multe verificări înainte de salvarea în baza de date.

```
public async Task<bool> SignUp(CreateUser request)
{
    if (await _context.Users.AnyAsync(u => u.UserName == request.UserName))
    {
        _logger.LogWarning("Sign-up failed: username '{UserName}' already exists.",
request.UserName);
        return false;
    }

    string salt = PasswordHelper.CreateSalt();
    string hash = PasswordHelper.HashPassword(request.Password, salt);

    var user = new User
    {
        Email = request.Email,
        UserName = request.UserName,
        Salt = salt,
        Hash = hash
    };

    await _context.Users.AddAsync(user);
    await _context.SaveChangesAsync();

    _logger.LogInformation("User '{UserName}' registered successfully.",
request.UserName);
    return true;
}
```

Primul pas constă în verificarea existenței unui utilizator cu același nume. Dacă numele de utilizator este deja folosit, operația este oprită, iar sistemul returnează răspuns negativ. Această verificare previne apariția duplicatelor și menține unicitatea conturilor.

Dacă utilizatorul nu există deja, sistemul generează un salt unic prin metoda CreateSalt(). Salt-ul reprezintă o valoare aleatoare adăugată parolei înainte de criptare. Rolul său este de a preveni atacurile bazate pe tabele predefinite și de a face imposibilă identificarea parolelor identice între utilizatori diferiți.

Ulterior, parola introdusă este procesată prin metoda `HashPassword()`, împreună cu salt-ul generat. Rezultatul este un hash criptografic care va fi salvat în baza de date în locul parolei reale.

Această abordare este extrem de importantă din punct de vedere al securității. În cazul unui acces neautorizat la baza de date, parolele reale nu pot fi citite direct, deoarece nu sunt stocate în clar. Sistemul păstrează doar hash-ul și salt-ul, iar la autentificare parola introdusă este rehash-uită și comparată cu valoarea existentă.

După generarea hash-ului, se creează obiectul `User` și se salvează în baza de date prin `Entity Framework Core`.

3.3.3 Configurarea JWT în fișierul `appsettings.json`

Pentru implementarea autentificării moderne bazate pe token-uri, în fișierul de configurare `appsettings.json` au fost definite proprietăți specifice JWT.

```
"Jwt": {  
  "Key": "E9vtP5VI1WKoKEw3CcOYmmsMgaXgLxvkv0KkPS20E4P",  
  "Issuer": "OurPlanApi",  
  "Audience": "OurPlanApiUsers",  
  "ExpireMinutes": 1440  
},
```

În această secțiune sunt definite următoarele elemente:

- `Key` – cheia secretă utilizată la semnarea token-ului
- `Issuer` – aplicația care emite token-ul
- `Audience` – destinatarii token-ului
- `ExpireMinutes` – durata de valabilitate a sesiunii

Cheia secretă este extrem de importantă, deoarece garantează autenticitatea token-ului și împiedică falsificarea acestuia.

3.3.4 Generarea Bearer Token-ului

După autentificarea cu succes a utilizatorului, sistemul generează automat un token JWT, care este transmis clientului și utilizat în sesiunile ulterioare.

```

private TokenResponse GenerateJwtToken(User user)
{
    TokenResponse response = new TokenResponse();
    var jwtSettings = _config.GetSection("Jwt");
    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtSettings["Key"]));

    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Email, user.Email),
        new Claim(ClaimTypes.Name, user.UserName)
    };

    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: jwtSettings["Issuer"],
        audience: jwtSettings["Audience"],
        claims: claims,
        expires:
DateTime.UtcNow.AddMinutes(Convert.ToDouble(jwtSettings["ExpireMinutes"])),
        signingCredentials: creds
    );

    response.Token = new JwtSecurityTokenHandler().WriteToken(token);
    return response;
}

```

Procesul începe prin încărcarea configurației JWT din fișierul appsettings.json. Apoi se creează cheia criptografică folosită la semnare.

În interiorul token-ului sunt introduse claim-uri relevante despre utilizator:

- identificator unic
- adresa de email
- numele utilizatorului

Aceste informații permit identificarea rapidă a utilizatorului la fiecare cerere, fără interogări suplimentare în baza de date.

Ulterior se definesc credențialele de semnare utilizând algoritmul HmacSha256 [19], unul dintre cele mai utilizate mecanisme pentru semnarea token-urilor JWT.

Token-ul este apoi generat specificând emitentul, audiența, datele utilizatorului și momentul expirării. La final, acesta este convertit într-un șir text și transmis clientului.

3.3.5 Importanța Bearer Token-ului în sesiunea utilizatorului

Bearer token-ul reprezintă cheia sesiunii utilizatorului în aplicațiile moderne de tip SPA (Single Page Application). După autentificare, token-ul este salvat în client și transmis în header-ul fiecărei cereri HTTP:

Ex:

```
{eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnR5Zy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1laWRlbnRpbmllciI6IjEiLCJodHRwOi8vc2NoZW1hcy54bWxzbnR5Zy...}
```

La primirea unei cereri, backend-ul verifică semnătura, valabilitatea și conținutul token-ului. Dacă acesta este valid, utilizatorul primește acces la resursele protejate.

Această metodă oferă mai multe avantaje în cadrul aplicațiilor web moderne. Prin utilizarea Bearer Token-ului, nu mai este necesară păstrarea unor sesiuni clasice pe server, deoarece informațiile necesare autentificării sunt transmise prin token la fiecare cerere. Acest lucru permite o scalare mai ușoară a aplicației și contribuie la creșterea performanței, deoarece serverul nu trebuie să gestioneze permanent starea fiecărui utilizator autentificat. De asemenea, utilizarea token-ului simplifică procesul de autentificare dintre frontend și backend, fiind o soluție des întâlnită în aplicațiile web moderne pentru protejarea endpoint-urilor și transmiterea sigură a identității utilizatorului.

În cadrul aplicației TravelS, token-ul este utilizat pentru accesarea funcțiilor private, precum salvarea preferințelor, gestionarea profilului și utilizarea resurselor rezervate utilizatorilor autentificați.

3.3.6 Implementarea principalelor entități ale sistemului

Pentru modelarea aplicației au fost create mai multe entități specializate. Entitatea Objective reprezintă unul dintre cele mai importante modele ale aplicației, deoarece stochează informațiile despre obiectivele turistice afișate utilizatorilor.

```

namespace TravelSBE.Entity
{
    public class Objective : BaseAuditEntity
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; set; }
        public string Name { get; set; }
        public string? Description { get; set; }
        public double Latitude { get; set; }
        public double Longitude { get; set; }
        public string? City { get; set; }
        [ForeignKey(nameof(ObjectiveType))]
        public int? Type { get; set; }
        public ObjectiveType? ObjectiveType { get; set; }
        public string? Website { get; set; }
        public string? Interval { get; set; }
        public string? Pret { get; set; }
        public int? Duration { get; set; }
        public Point? Location { get; set; }

        public List<ObjectiveImage> Images { get; set; } = new
List<ObjectiveImage>();
        public List<Review> Reviews { get; set; } = new List<Review>();
    }
}

```

3.3.7 Relația dintre Objective și ObjectiveType

Fiecare obiectiv turistic poate aparține unei categorii. Pentru acest scop a fost creată entitatea ObjectiveType, utilizată pentru clasificări precum: muzee, parcuri, monumente, biserici, etc.

În clasa Objective există proprietatea:

```

[ForeignKey(nameof(ObjectiveType))]
public int? Type { get; set; }
public ObjectiveType? ObjectiveType { get; set; }

```

Aceasta indică faptul că fiecare obiectiv poate avea asociat un tip. În baza de date, coloana Type devine foreign key către tabela ObjectiveTypes.

3.3.8 Relația dintre Objective și ObjectiveImage

Un obiectiv turistic poate avea mai multe imagini asociate. Din acest motiv s-a implementat relația de tip one-to-many între Objective și ObjectiveImage.

În cadrul contextului de baza de date, această relație a fost definită astfel:

```
builder.Entity<Objective>()  
    .HasMany(e => e.Images)  
    .WithOne(e => e.Objective)  
    .HasForeignKey(e => e.IdObjective);
```

Această configurație înseamnă că:

- un obiectiv poate avea mai multe imagini
- fiecare imagine aparține unui singur obiectiv
- legătura este realizată prin coloana IdObjective

Această structură permite afișarea galeriilor foto pentru fiecare obiectiv turistic.

3.3.9 Relația dintre Objective și Review

Pentru creșterea interactivității aplicației, utilizatorii pot lăsa recenzii pentru obiectivele turistice. Astfel, a fost implementată entitatea Review.

Relația este tot de tip one-to-many:

```
builder.Entity<Objective>()  
    .HasMany(e => e.Reviews)  
    .WithOne(e => e.Objective)  
    .HasForeignKey(e => e.IdObjective)  
    .onDelete(DeleteBehavior.Cascade);
```

Această relație arată că un obiectiv turistic poate avea mai multe recenzii asociate, în timp ce fiecare recenzie aparține unui singur obiectiv. Prin această structură, aplicația poate afișa toate evaluările și comentariile primite pentru un anumit obiectiv. De asemenea, în cazul în care un obiectiv este șters din baza de date, recenziile asociate acestuia se șterg automat, pentru a evita păstrarea unor date care nu mai au legătură cu o înregistrare existentă.

Utilizarea DeleteBehavior.Cascade menține consistența datelor și previne existența recenziilor fără obiectiv asociat.

3.3.10 Implementarea entităților Event, Itinerary și ItineraryDetails

După definirea entităților principale ale aplicației, precum utilizatorii, obiectivele turistice, tipurile de obiective, imaginile și recenziile, dezvoltarea sistemului a continuat prin adăugarea unor modele suplimentare, necesare pentru extinderea funcționalităților platformei TravelS. Aceste entități au fost introduse cu scopul de a transforma aplicația dintr-un simplu sistem de afișare a obiectivelor turistice într-o platformă mai complexă, capabilă să sprijine utilizatorul în procesul de informare, planificare și organizare a unei vizite.

Una dintre aceste entități este Event, utilizată pentru stocarea informațiilor despre evenimente locale, festivaluri, concerte, târguri sau alte activități organizate într-o anumită zonă turistică. Integrarea acestei entități este importantă deoarece experiența turistică nu se limitează doar la vizitarea obiectivelor statice, ci include și participarea la activități desfășurate într-un anumit interval de timp. Prin intermediul acestei structuri, aplicația poate afișa evenimente relevante pentru utilizatori, împreună cu informații precum denumirea, descrierea, locația, data de început, data de finalizare și eventualele date asociate obiectivului sau zonei în care se desfășoară.

Entitatea Itinerary a fost creată pentru a permite reprezentarea itinerariilor definite de utilizatori. Un itinerariu poate fi înțeles ca un plan de vizitare care grupează mai multe obiective turistice într-o structură organizată. Această funcționalitate este utilă în special pentru utilizatorii care doresc să își planifice o excursie sau o zi de vizitare, selectând din aplicație obiectivele care îi interesează și reunindu-le într-un traseu personalizat.

Pentru realizarea legăturii dintre itinerarii și obiectivele incluse în acestea a fost introdusă entitatea ItineraryDetails. Aceasta are rolul de tabel de legătură și permite asocierea mai multor obiective sau evenimente cu un itinerariu. În plus, prin intermediul acestei entități poate fi păstrată ordinea de vizitare, ceea ce oferă utilizatorului posibilitatea de a construi trasee personalizate, adaptate preferințelor sale și timpului disponibil.

Această structură contribuie la flexibilitatea aplicației, deoarece separă informațiile generale despre itinerariu de elementele concrete care îl compun. Astfel, un itinerariu poate conține mai multe obiective, iar același obiectiv poate fi inclus în itinerarii diferite, create de utilizatori diferiți. Această abordare este potrivită pentru o aplicație turistică modernă, întrucât permite planificarea călătoriilor într-un mod organizat și personalizat.

Prin implementarea entităților Event, Itinerary și ItineraryDetails, aplicația TravelS a fost extinsă cu funcționalități care depășesc simpla prezentare a obiectivelor turistice. Utilizatorul poate consulta evenimente locale, poate crea planuri de vizitare și poate organiza obiectivele într-o succesiune logică. Aceste modele contribuie astfel la dezvoltarea unei experiențe turistice mai complete, orientate atât spre descoperirea locațiilor, cât și spre planificarea efectivă a activităților.

3.3.11 Implementarea operațiilor CRUD

După definirea entităților principale ale aplicației și configurarea relațiilor dintre acestea, următorul pas important în dezvoltarea backend-ului a fost implementarea operațiilor CRUD. Aceste operații reprezintă funcționalitățile de bază prin care aplicația poate gestiona datele stocate în baza de date și permit realizarea completă a fluxurilor de administrare pentru entitățile sistemului.

Acronimul CRUD provine de la cele patru operații fundamentale utilizate în lucrul cu datele:

- Create – adăugarea unor înregistrări noi în baza de date;
- Read – citirea și afișarea informațiilor existente;
- Update – modificarea datelor deja salvate;
- Delete – ștergerea unor resurse din sistem.

În cadrul aplicației TravelS, aceste operații au fost implementate pentru entitățile principale ale sistemului, precum Objective, ObjectiveType, Event, Itinerary, ItineraryDetails, Review și ObjectiveImage. Prin intermediul acestor operații, aplicația poate administra conținutul afișat utilizatorilor și permite actualizarea permanentă a informațiilor turistice disponibile în platformă.

Implementarea operațiilor CRUD a fost realizată printr-o separare clară între controller și service. Controller-ul are rolul de a primi cererile HTTP venite din frontend, de a prelua parametrii transmiși de utilizator și de a apela metoda corespunzătoare din service. Service-ul conține logica aplicației, validează datele primite, interacționează cu baza de date prin intermediul ApplicationDbContext și returnează rezultatul operației către controller.

Pentru exemplificarea modului de funcționare, poate fi analizată entitatea Objective, una dintre cele mai importante entități ale aplicației. Aceasta stochează informațiile despre obiectivele turistice, precum denumirea, descrierea, coordonatele geografice, orașul, tipul obiectivului, site-ul web, prețul, durata estimată a vizitei și datele utilizate ulterior în algoritmul de recomandare.

Pentru această entitate au fost implementate endpoint-uri care permit:

- adăugarea unui obiectiv nou;
- afișarea tuturor obiectivelor;
- afișarea detaliilor unui obiectiv;
- modificarea informațiilor unui obiectiv existent;
- ștergerea unui obiectiv.

Operația de adăugare a unui obiectiv nou corespunde acțiunii Create. Aceasta este declanșată atunci când administratorul completează formularul de adăugare din interfața aplicației și trimite datele către backend.

```
public async Task<ServiceResult<ObjectiveModel>> CreateObjectiveAsync(ObjectiveModel
objective)
{
    ServiceResult<ObjectiveModel> result = new();
    if (String.IsNullOrEmpty(objective.Name) && String.IsNullOrEmpty(objective.City)
&& objective.Latitude == null && objective.Longitude == null)
    {
        result.ValidationMessage = "Nu ati completat toate campurile pentru a putea
adauga un obiectiv";
        result.IsSuccessful = false;
        return result;
    }

    if (string.IsNullOrEmpty(objective.Description))
    {
        objective.Description = await GenerateObjectiveDetailsAsync(objective.Name,
objective.City);
    }

    objective.CreatedAt = DateTime.UtcNow;
    objective.UpdatedAt = DateTime.UtcNow;

    var mapped = _mapper.Map<Objective>(objective);

    mapped.Location = new Point(objective.Longitude, objective.Latitude) { SRID =
4326 };

    _context.Objectives.Add(mapped);
    await _context.SaveChangesAsync();

    await _mlService.TrainModelAsync();
}
```



```

        await _mlService.UpdateClusterNeighborsAsync();

        result.Result = _mapper.Map<ObjectiveModel>(mapped);
        return result;
    }

```

La nivel conceptual, operația de adăugare presupune următorii pași: primirea datelor din frontend, validarea acestora, construirea obiectului de tip Objective, apelarea metodei Add sau AddAsync din Entity Framework Core și salvarea modificărilor prin SaveChangesAsync. După salvare, noua înregistrare devine disponibilă în baza de date și poate fi afișată în interfața aplicației.

Operația de citire a informațiilor corespunde acțiunii Read. Aceasta este utilizată atât pentru afișarea listei complete de obiective turistice, cât și pentru afișarea detaliilor unui singur obiectiv.

```

public async Task<ServiceResult<ObjectiveModel>> GetObjectiveByIdAsync(int id)
{
    var result = new ServiceResult<ObjectiveModel>();

    var item = await _context.Objectives
        .AsNoTracking()

        .Include(x => x.ObjectiveType)
        .Include(x => x.Images)
        .Include(x => x.Reviews)
        .FirstOrDefaultAsync(x => x.Id == id);

    if (item == null)
    {
        result.ValidationMessage = "Objective not found.";
        return result;
    }

    var objectiveModel = _mapper.Map<ObjectiveModel>(item);
    objectiveModel.Images = item.Images
        .Select(img => $"{_baseUrl}{img.FilePath}")
        .ToList();

    if (item.Reviews.Any())
    {
        objectiveModel.MedieReview = item.Reviews.Average(r => r.Raiting);
    }
    else
    {
        objectiveModel.MedieReview = null;
    }

    result.Result = objectiveModel;
    return result;
}

```

Această operație este importantă deoarece majoritatea paginilor din aplicație depind de citirea datelor din backend. De exemplu, pagina principală a obiectivelor turistice, pagina de

detaliu, modulul de administrare și sistemul de recomandare utilizează informațiile obținute din baza de date.

Operația de actualizare corespunde acțiunii Update. Aceasta este utilizată atunci când administratorul modifică informațiile unui obiectiv existent.

```
public async Task<ServiceResult<ObjectiveModel>> UpdateObjectiveAsync(Objective
objective)
{
    ServiceResult<ObjectiveModel> result = new();
    if (!_context.Objectives.Any(e => e.Id == objective.Id))
    {
        return new();
    }

    objective.UpdatedAt = DateTime.UtcNow;
    _context.Entry(objective).State = EntityState.Modified;
    await _context.SaveChangesAsync();
    result.Result = _mapper.Map<ObjectiveModel>(objective);
    return result;
}
```

La nivel logic, actualizarea presupune identificarea resursei existente, validarea datelor noi, modificarea proprietăților entității și salvarea schimbărilor. Dacă obiectivul nu este găsit în baza de date, serverul poate returna un răspuns de tip 404 Not Found. Dacă modificarea este realizată cu succes, aplicația poate returna un răspuns de confirmare către frontend.

Operația de ștergere corespunde acțiunii Delete. Aceasta este utilizată atunci când administratorul dorește eliminarea unui obiectiv din sistem.

```
public async Task<ServiceResult<bool>> DeleteObjectiveAsync(int id)
{
    ServiceResult<bool> result = new();
    var objective = await _context.Objectives.FindAsync(id);
    if (objective == null)
    {
        result.Result = false;
    }

    _context.Objectives.Remove(objective);
    await _context.SaveChangesAsync();
    result.Result = true;
    return result;
}
```

Ștergerea trebuie tratată cu atenție deoarece anumite entități pot avea relații cu alte tabele. De exemplu, un obiectiv poate avea imagini, recenzii sau poate fi inclus într-un itinerariu. Din acest motiv, relațiile dintre entități și comportamentul la ștergere au fost configurate astfel încât baza de date să rămână într-o stare consistentă.

Acceași logică de implementare a fost aplicată și pentru celelalte entități ale sistemului. Pentru Event au fost create operații prin care pot fi adăugate, afișate, modificate sau șterse

evenimente locale. Pentru `ObjectiveType` au fost implementate operații de administrare a categoriilor de obiective turistice. Pentru `Itinerary` și `ItineraryDetails`, operațiile CRUD permit crearea și gestionarea itinerariilor personalizate. În cazul entității `Review`, operațiile permit utilizatorilor să adauge și să consulte recenzii asociate obiectivelor turistice.

Prin implementarea operațiilor CRUD complete, aplicația oferă un mecanism eficient de administrare a conținutului. Administratorul poate menține informațiile actualizate, poate adăuga obiective noi, poate corecta date existente și poate elimina informațiile care nu mai sunt relevante. În același timp, aceste operații reprezintă baza funcțională pe care sunt construite celelalte componente ale aplicației, precum afișarea obiectivelor în interfață, sistemul de filtrare, modulul de administrare și mecanismul de recomandare.

Astfel, operațiile CRUD nu reprezintă doar funcții tehnice de bază, ci elemente esențiale pentru funcționarea întregii platforme. Fără acestea, aplicația nu ar putea gestiona dinamic informațiile turistice și nu ar putea oferi utilizatorilor conținut actualizat și relevant.

3.3.12 Menținerea fiabilității codului

Fiabilitatea codului reprezintă un aspect important în dezvoltarea aplicației, deoarece influențează stabilitatea, mentenabilitatea și capacitatea sistemului de a funcționa corect în diferite scenarii de utilizare. În cadrul aplicației `TravelS`, fiabilitatea a fost urmărită prin organizarea clară a logicii aplicației în servicii, modele, controllere și componente auxiliare. Această separare a responsabilităților permite o mai bună structurare a codului, facilitează verificarea fiecărei componente și reduce riscul apariției erorilor în momentul extinderii sau modificării funcționalităților existente.

Pentru validarea comportamentului aplicației au fost introduse teste unitare, organizate într-un proiect separat, denumit `TravelsBE.Tests`, inclus în soluția principală. Această abordare reflectă modul de lucru utilizat în proiectele software de dimensiuni mari, unde codul aplicației și codul destinat testării sunt separate, dar integrate în același ecosistem de dezvoltare. Prin această organizare, testele pot fi rulate independent de aplicația principală, oferind o metodă rapidă și controlată de verificare a corectitudinii anumitor funcționalități.

Testele unitare au rolul de a verifica funcționalități izolate ale aplicației, fără a depinde de infrastructura externă, precum baza de date PostgreSQL sau serviciile de machine learning. Pentru realizarea acestui lucru a fost utilizată o bază de date în memorie, prin intermediul Entity

Framework Core InMemory. Aceasta permite simularea operațiilor asupra datelor fără conectarea la o bază de date reală. De asemenea, serviciile externe sau componentele care nu sunt relevante pentru scenariul testat au fost înlocuite cu implementări false, controlate direct în cadrul testelor. Astfel, testele rulează rapid, sunt predictibile și pot identifica mai ușor eventualele regresii apărute în logica aplicației.

Un exemplu relevant este testarea metodei de autentificare din cadrul serviciului UserService. Testul creează un utilizator într-o bază de date în memorie, salvează parola acestuia sub formă de hash, apoi verifică dacă metoda Login returnează un token JWT doar în situația în care datele de autentificare introduse sunt corecte.

```
[Fact]
public async Task Login_ReturnsJwtToken_WhenCredentialsAreValid()
{
    await using var context = CreateContext();
    var user = await SeedUser(context, "ana", "Parola123!", "ana@example.com");
    var service = CreateService(context);

    var response = await service.Login("ana", "Parola123!");

    Assert.False(string.IsNullOrEmpty(response.Token));

    var jwt = new JwtSecurityTokenHandler().ReadJwtToken(response.Token);
    Assert.Equal(user.Id.ToString(), jwt.Claims.Single(c => c.Type ==
ClaimTypes.NameIdentifier).Value);
    Assert.Equal("ana@example.com", jwt.Claims.Single(c => c.Type ==
ClaimTypes.Email).Value);
    Assert.Equal("ana", jwt.Claims.Single(c => c.Type == ClaimTypes.Name).Value);
}
```

Acest test demonstrează faptul că mecanismul de autentificare funcționează corect pentru un utilizator valid. Verificarea nu se limitează doar la existența tokenului JWT, ci urmărește și corectitudinea informațiilor incluse în acesta. Astfel, testul confirmă faptul că tokenul generat conține identificatorul utilizatorului, adresa de e-mail și numele de utilizator corespunzătoare contului autentificat.

Prin rularea automată a acestui tip de test, eventualele modificări ulterioare care afectează procesul de autentificare pot fi detectate rapid. În acest mod, testele unitare contribuie la creșterea fiabilității aplicației, oferind un mecanism de verificare constantă a funcționalităților esențiale și reducând riscul introducerii unor erori în timpul dezvoltării.

3.4 Implementarea frontend-ului în Vue.js

Implementarea frontend-ului aplicației TravelS a reprezentat etapa în care funcționalitățile dezvoltate la nivel de backend au fost transpuse într-o interfață vizuală interactivă, accesibilă utilizatorilor finali. Într-o aplicație modernă, frontend-ul nu mai are doar

rolul de a afișa date, ci gestionează interacțiunea utilizatorului cu sistemul, oferă feedback în timp real și contribuie semnificativ la experiența generală de utilizare. Din acest motiv, alegerea tehnologiilor și modul de organizare a interfeței au fost realizate cu atenție, urmărindu-se obținerea unei aplicații fluide, intuitive și ușor de extins.

3.4.1 Configurarea proiectului cu Vite

Dezvoltarea frontend-ului a început prin inițializarea proiectului utilizând Vite [20], o librărie utilă de build, recunoscută pentru viteza ridicată și eficiența în timpul dezvoltării. Alegerea Vite a fost determinată de timpul foarte scurt de pornire a aplicației și de mecanismul de actualizare instantanee a modificărilor, fără reîncărcarea completă a paginii. Acest lucru a contribuit semnificativ la eficientizarea procesului de dezvoltare.

```
npm create vite@latest
```

După crearea proiectului de bază, au fost instalate dependențele necesare pentru dezvoltarea interfeței. Printre acestea se numără:

- PrimeVue – bibliotecă de componente UI moderne
- Vue Router – pentru gestionarea rutelor și navigației
- PrimeFlex – librărie CSS pentru crearea layout-ului

Acestea au oferit un set complet de componente și funcționalități, reducând semnificativ timpul necesar implementării interfeței.

3.4.2 Implementarea paginii de autentificare

Primul modul dezvoltat în frontend a fost pagina de autentificare. Aceasta permite utilizatorilor să își creeze conturi noi sau să se autentifice în aplicație utilizând datele existente.

Interfața a fost construită utilizând componente PrimeVue pentru formulare, câmpuri de input și butoane. La trimiterea formularului, datele sunt transmise către backend, unde sunt validate și procesate. În cazul autentificării reușite, utilizatorul primește un token JWT, care este salvat la nivelul aplicației (local storage) și utilizat în cererile ulterioare.

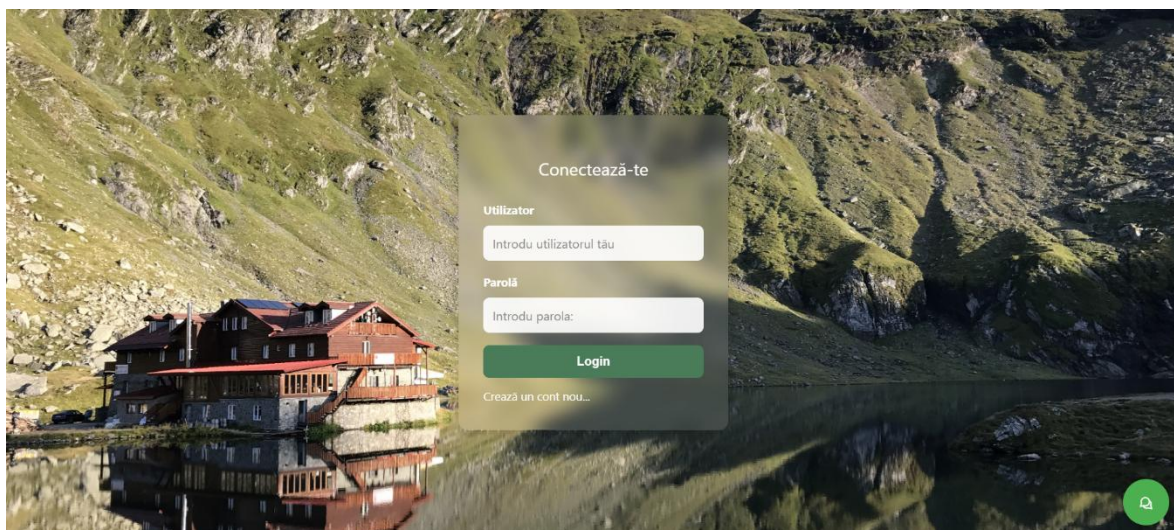


Figura 12 – Pagina de autentificare

Pagina de autentificare (Figura 12), reprezintă punctul de intrare în aplicație și asigură securizarea accesului la funcționalitățile disponibile.

3.4.3 Implementarea paginii pentru afișarea obiectivelor turistice

După finalizarea autentificării, dezvoltarea a continuat cu pagina principală pentru afișarea obiectivelor turistice (Figura 13). Aceasta reprezintă una dintre cele mai importante componente ale aplicației, deoarece oferă utilizatorilor acces rapid la informațiile principale despre locațiile disponibile în platformă. Interfața aplicației include un navigation bar prin intermediul căruia utilizatorul poate accesa principalele secțiuni ale sistemului, precum obiectivele turistice, experiențele, evenimentele, itinerariile, modulul de administrare și profilul utilizatorului.

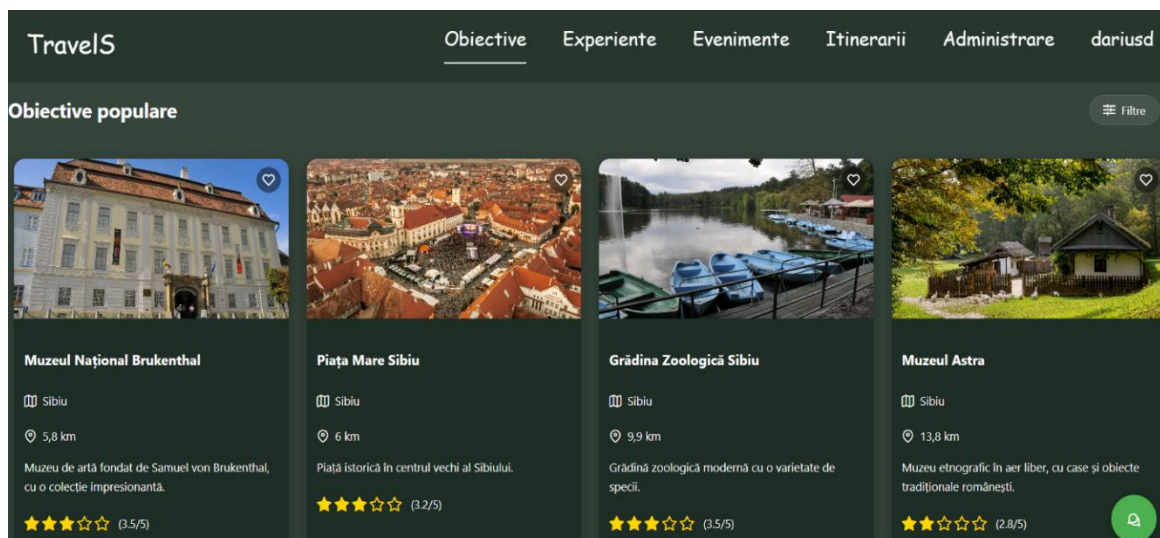


Figura 13 – Pagina pentru afișarea obiectivelor turistice

În cadrul paginii dedicate obiectivelor turistice, utilizatorul are posibilitatea de a filtra rezultatele după diferite proprietăți, cum ar fi denumirea obiectivului, tipul acestuia, distanța maximă față de locația curentă a utilizatorului și ratingul minim (Figura 13). Aceste filtre contribuie la o experiență mai personalizată, deoarece permit afișarea doar a obiectivelor relevante pentru preferințele utilizatorului. Sub zona de filtrare este afișată lista obiectivelor turistice disponibile, fiecare obiectiv putând fi selectat pentru vizualizarea detaliilor complete, precum descrierea, imaginile, locația, informațiile suplimentare și recenziile asociate.

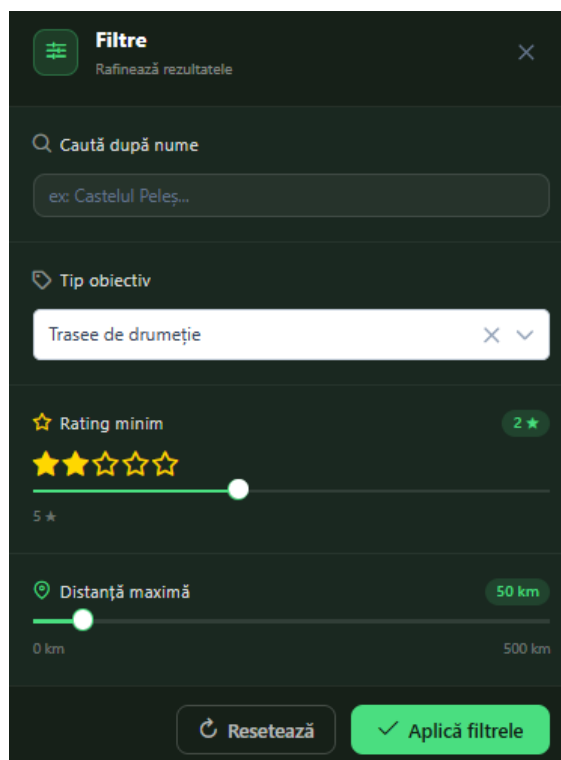


Figura 14 – Componenta pentru filtrarea obiectivelor

Pentru a putea dezvolta această funcționalitate, datele necesare au fost introduse inițial manual în baza de date. Această abordare a permis modelarea structurii entităților și testarea rapidă a interfeței, fără a depinde de un modul complet de administrare. Pe măsură ce aplicația a evoluat, această metodă manuală a devenit inefficientă, ceea ce a condus la necesitatea dezvoltării unui modul dedicat de administrare.

3.4.4 Modulul de administrare

Pentru gestionarea datelor din aplicație a fost implementat un modul de administrare, destinat utilizatorilor cu roluri speciale (Figura 15). Acest modul permite efectuarea operațiilor CRUD asupra entităților principale.

Interfața acestui modul a fost realizată sub formă tabelară, utilizând componentele oferite de PrimeVue. Fiecare tabel permite vizualizarea datelor existente, precum și adăugarea, editarea și ștergerea acestora.



Gestionează obiectivele						
Caută după nume						
Id	Nume	Oraș	Descriere	Latitudine	Longitudine	Acțiuni
17	Mănăstirea Curtea de Argeș	Curtea de Argeș	Una dintre cele mai frumoase mănăstiri din România.	45.1489	24.6793	  
38	Mănăstirea Săpânța-Peri	Săpânța	Cea mai înaltă biserică de lemn din lume.	47.9773	23.6875	  
41	Cascada Cailor	Borșa	Cea mai mare cascadă din România.	47.6103	24.7379	  

Figura 15 – Modulul de administrare

Pentru operațiile de adăugare și editare au fost create formulare dedicate, în care utilizatorul poate introduce sau modifica informațiile asociate fiecărei entități.

Un aspect important în cadrul acestor formulare îl reprezintă setarea locației geografice. Pentru acest scop a fost utilizată biblioteca open-source OpenLayers, prezent în Figura16, care permite afișarea unei hărți interactive și selectarea unui punct geografic.

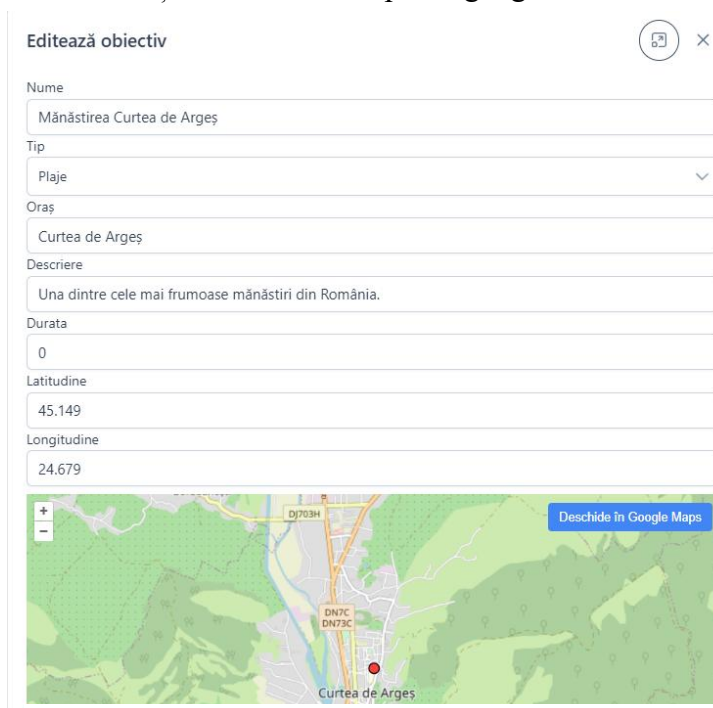


Figura 16 – Formularul de editare al unui obiectiv turistic

Prin interacțiunea cu harta, utilizatorul poate selecta poziția exactă a unui obiectiv, eveniment sau experiență, iar coordonatele sunt salvate automat în baza de date.

3.4.5 Implementarea paginii de detaliu pentru obiectiv

După finalizarea modului de administrare, dezvoltarea frontend-ului a continuat cu implementarea paginii de detaliu pentru un obiectiv turistic prezentată în Figura 17. Această pagină oferă o prezentare completă a informațiilor asociate unei entități și reprezintă unul dintre cele mai importante puncte de interacțiune pentru utilizator.

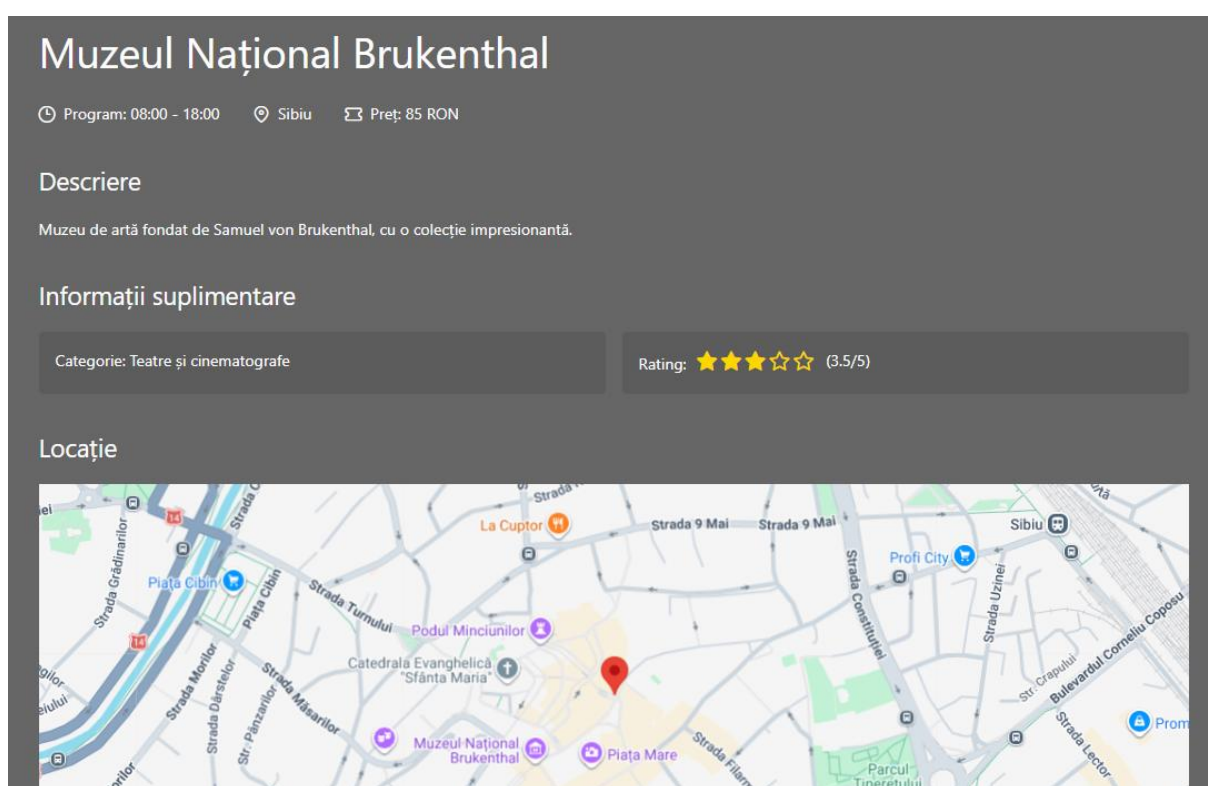


Figura 17 – Pagina de detaliu al unui obiectiv

Pentru afișarea imaginilor asociate obiectivului a fost utilizat un slider, care permite parcurgerea acestora în mod dinamic. În cazul în care există mai multe imagini, utilizatorul poate naviga între ele fără a părăsi pagina (Figura 18).

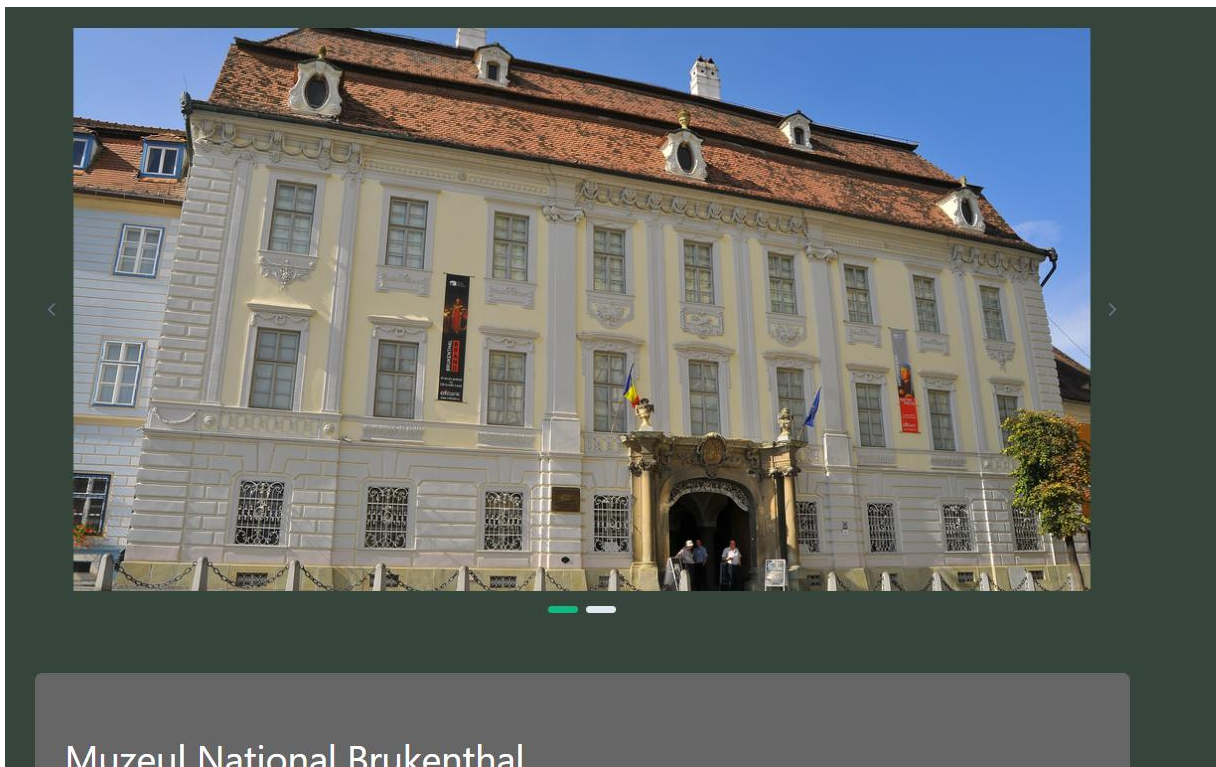


Figura 18 – Slider pentru vizualizarea imaginilor

Pe lângă imagini, pagina conține informații detaliate precum:

- descrierea obiectivului
- prețul de intrare
- categoria (tipul obiectivului)
- durata estimată a vizitei
- rating-ul mediu
- recenziile utilizatorilor

Locația obiectivului este reprezentată prin integrarea unui iframe cu Google Maps, care oferă o vizualizare familiară și ușor de utilizat.

Recenziile adăugate de utilizatori sunt afișate într-o secțiune dedicată, contribuind la crearea unei experiențe interactive și la oferirea de feedback real pentru ceilalți utilizatori.

3.4.6 Dezvoltarea celorlalte pagini

După finalizarea paginii pentru obiective, au fost implementate și celelalte componente ale interfeței, precum paginile pentru evenimente, experiențe și itinerarii. Acestea au fost dezvoltate utilizând aceeași structură și aceleași principii de design, pentru a menține consistența aplicației.

Fiecare pagină oferă funcționalități similare: listare, filtrare, vizualizare detalii și interacțiune cu utilizatorul. Această uniformitate contribuie la o experiență de utilizare coerentă și intuitivă.

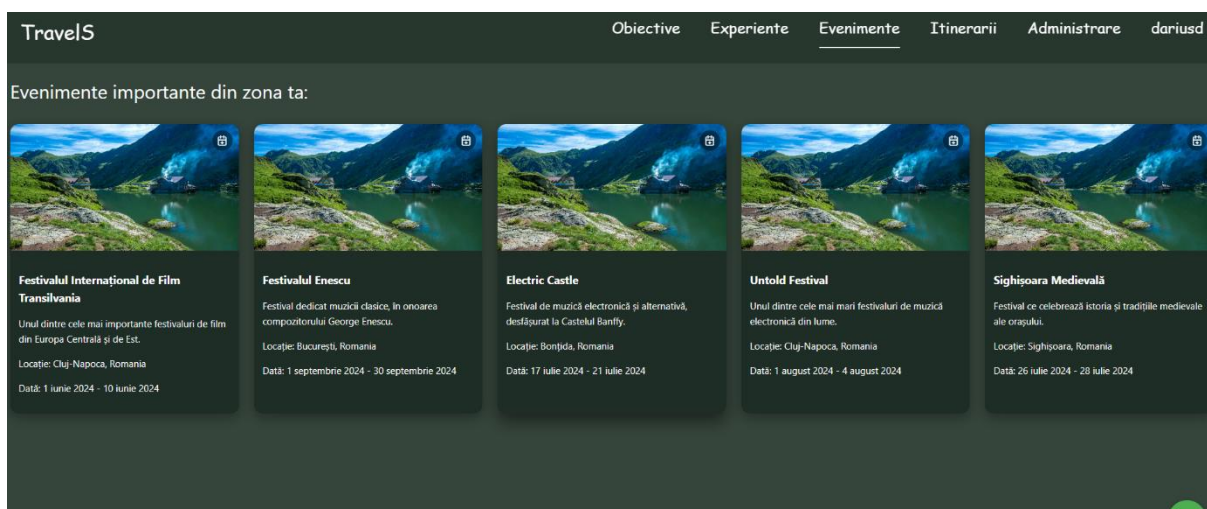


Figura 19 – Pagina de evenimente

3.5 Algoritmul de recomandare

Un element important al aplicației TravelS este reprezentat de mecanismul de recomandare a obiectivelor turistice. Acesta are rolul de a sprijini utilizatorul în procesul de descoperire a unor locații relevante, pe baza caracteristicilor obiectivelor existente în aplicație. În loc ca utilizatorul să caute manual fiecare obiectiv, sistemul poate identifica automat obiective asemănătoare, ținând cont de criterii precum poziția geografică, tipul obiectivului, rating-ul și prețul estimativ.

Pentru realizarea acestui mecanism a fost utilizată o abordare bazată pe clusterizare, prin care obiectivele turistice sunt grupate în funcție de similaritatea dintre ele. Astfel, obiectivele care prezintă caracteristici apropiate sunt încadrate în același grup, iar aceste grupuri

pot fi folosite ulterior pentru generarea recomandărilor. Această metodă permite oferirea unor sugestii mai relevante și contribuie la îmbunătățirea experienței utilizatorului în cadrul aplicației.

În cadrul acestui subcapitol sunt prezentate etapele principale ale algoritmului de recomandare, începând cu încărcarea și pregătirea datelor, continuând cu normalizarea caracteristicilor și aplicarea algoritmului de clusterizare, iar în final cu determinarea obiectivelor similare care pot fi recomandate utilizatorului.

3.5.1 Încarcarea obiectivelor

Prima etapă a algoritmului de recomandare constă în colectarea și pregătirea datelor care urmează să fie procesate în cadrul mecanismului de clusterizare. Această etapă are un rol esențial deoarece calitatea și structura datelor influențează în mod direct performanța algoritmului și relevanța recomandărilor generate ulterior.

În cadrul aplicației, obiectivele turistice reprezintă elementele principale analizate de algoritm. Fiecare obiectiv conține un set de informații relevante care descriu caracteristicile acestuia și care permit compararea cu alte locații similare. Printre aceste informații se numără coordonatele geografice, categoria obiectivului, rating-ul oferit de utilizatori și costul estimativ al accesului.

Înainte ca algoritmul să poată realiza procesul de clusterizare, toate aceste informații trebuie extrase și transformate într-o formă numerică standardizată. Acest lucru este necesar deoarece algoritmii de machine learning operează asupra valorilor matematice și nu pot interpreta direct informații textuale sau structuri complexe de date.

Procesul de pregătire a datelor începe prin extragerea tuturor obiectivelor turistice din baza de date. Pentru fiecare obiectiv turistic sunt colectate mai multe caracteristici principale, care au rolul de a descrie cât mai clar proprietățile acestuia și de a permite compararea sa cu alte obiective similare. Printre aceste caracteristici se numără coordonatele geografice, utilizate pentru determinarea poziției obiectivului și pentru calcularea distanței față de utilizator sau față de alte locații. De asemenea, este luat în considerare tipul obiectivului, care permite încadrarea acestuia într-o anumită categorie turistică. Rating-ul mediu reflectă aprecierea utilizatorilor asupra obiectivului, iar prețul estimativ oferă o indicație asupra costului asociat vizitării acestuia.

Coordonatele geografice sunt reprezentate prin valorile latitudine și longitudine și permit identificarea poziției obiectivului pe hartă. Proximitatea geografică reprezintă unul dintre cele mai importante criterii utilizate în procesul de recomandare.

Tipul obiectivului este utilizat pentru gruparea locațiilor similare din punct de vedere tematic. Astfel, muzeele, monumentele istorice, parcurile sau atracțiile naturale pot fi analizate separat și comparate în funcție de categoria din care fac parte.

Rating-ul mediu reprezintă evaluarea generală oferită de utilizatori și reflectă gradul de apreciere al unui obiectiv turistic. Introducerea acestui criteriu în algoritm permite prioritizarea locațiilor considerate mai atractive de către utilizatori.

Prețul estimativ contribuie la diferențierea obiectivelor din punct de vedere al accesibilității economice și permite identificarea unor recomandări mai apropiate de preferințele utilizatorilor.

```
var points = objectives.Select(o => new ClusteringPoint
{
    Objective = o,
    X = o.Location?.X ?? 0,
    Y = o.Location?.Y ?? 0,
    AverageRating = o.Reviews.Any() ? o.Reviews.Average(r => r.Raitng) : 0,
    Price = !string.IsNullOrEmpty(o.Pret) && int.TryParse(o.Pret, out var p) ? p : 0,
    TypeId = o.Type ?? 0
}).ToList();
```

După colectarea datelor, acestea sunt transformate într-o reprezentare numerică multidimensională. Fiecare obiectiv este reprezentat sub forma unui punct într-un spațiu matematic:

$p=(x,y, rating, price, type)$

unde:

- x și y reprezintă coordonatele geografice;
- $rating$ reprezintă evaluarea medie;
- $price$ reprezintă costul estimativ;
- $type$ reprezintă categoria obiectivului.

Această reprezentare permite algoritmului să compare obiectivele între ele și să identifice gradul de similaritate dintre acestea.

Un aspect important al etapei de pregătire îl reprezintă validarea datelor. În practică, anumite obiective pot avea informații incomplete, valori lipsă sau date inconsistente. Pentru evitarea problemelor în timpul procesului de clusterizare, sistemul verifică existența valorilor necesare și aplică valori implicite acolo unde este necesar.

De asemenea, în această etapă sunt calculate anumite valori suplimentare utilizate ulterior de algoritm, precum media rating-urilor sau transformarea prețurilor în valori numerice comparabile.

Procesul de pregătire a datelor reprezintă fundamentul întregului sistem de recomandare. O reprezentare corectă și coerentă a informațiilor permite algoritmului să identifice relații relevante între obiectivele turistice și contribuie la generarea unor recomandări mai precise și mai utile pentru utilizatori.

3.5.2 Normalizarea datelor

După colectarea și pregătirea informațiilor necesare, următoarea etapă importantă în funcționarea algoritmului de recomandare este reprezentată de normalizarea datelor. Această etapă are rolul de a aduce toate caracteristicile utilizate într-un interval comun de valori, astfel încât fiecare atribut să contribuie într-un mod echilibrat la procesul de clusterizare.

În cadrul aplicației TravelS, obiectivele turistice sunt descrise prin caracteristici foarte diferite din punct de vedere al valorilor numerice. De exemplu, coordonatele geografice pot avea valori mari și variații semnificative, deoarece acestea descriu poziționarea obiectivelor în spațiu prin latitudine și longitudine. În schimb, rating-ul utilizatorilor este limitat într-un interval redus, de regulă între 1 și 5, ceea ce face ca influența sa numerică să fie diferită față de celelalte caracteristici. Prețurile estimative pot varia de la valori foarte mici până la valori ridicate, în funcție de tipul obiectivului și de costurile asociate vizitării acestuia. De asemenea, tipul obiectivului este reprezentat numeric prin identificatori ai categoriilor, ceea ce permite includerea acestei informații în procesul de analiză și comparare a obiectivelor turistice.

Fără aplicarea unui proces de normalizare, caracteristicile cu valori mai mari ar influența disproorționat algoritmul. Într-un astfel de scenariu, coordonatele geografice ar domina procesul de calcul al distanțelor, iar celelalte criterii precum rating-ul sau categoria obiectivului ar avea o influență foarte redusă asupra rezultatului final.

Pentru evitarea acestei probleme, fiecare caracteristică este transformată într-o valoare standardizată utilizând o formulă de normalizare de tip Min-Max:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} * \text{Weight}$$

Figura 20 – Normalizare Min-Max

unde:

- x reprezintă valoarea inițială;
- xmin reprezintă valoarea minimă;
- xmax reprezintă valoarea maximă;
- xnorm reprezintă valoarea normalizată.

Prin aplicarea acestei formule, toate valorile sunt aduse într-un interval standardizat, de regulă între 0 și 1. În acest mod, fiecare caracteristică poate participa în mod echilibrat la procesul de calcul al similarității dintre obiective.

Procesul presupune identificarea valorilor minime și maxime pentru fiecare caracteristică în parte, urmată de transformarea fiecărei valori într-o variantă normalizată.

Un aspect important al implementării îl reprezintă utilizarea ponderilor pentru fiecare caracteristică. În practică, nu toate informațiile au aceeași importanță în procesul de recomandare. De exemplu, în cazul unei aplicații turistice, proximitatea geografică reprezintă un criteriu mult mai important decât diferențele mici de preț sau rating.

Din acest motiv, după procesul de normalizare, fiecare caracteristică este multiplicată cu o pondere specifică:

$$\text{ValoareFinala} = \text{ValoareNormalizata} \times \text{Pondere}$$

Locația geografică are cea mai mare pondere, deoarece obiectivele apropiate sunt considerate mai relevante pentru utilizator. Rating-ul și prețul au ponderi mai reduse, iar categoria obiectivului contribuie la identificarea locațiilor similare din punct de vedere tematic.

Această abordare permite ajustarea comportamentului algoritmului și oferă flexibilitate în modul de generare a recomandărilor. Astfel, sistemul poate prioritiza anumite caracteristici în funcție de importanța lor în contextul aplicației.

După finalizarea procesului de normalizare, fiecare obiectiv turistic este reprezentat printr-un set de valori standardizate și ponderate, pregătite pentru etapa următoare a algoritmului: procesul efectiv de clusterizare.

normalizedPoints	Count = 101	View
> [0]	(0.16122463359639233, 0.40992113564668758, 0, 0.008, 0.0125)	
> [1]	(0.20589064261555809, 0.25594637223974775, 0, 0.06899999999999992, 0.0625)	
> [2]	(0.38519588500563695, 0.17981072555205044, 0, 0.053000000000000005, 0.1)	
> [3]	(0.34182638105975183, 0.047097791798107295, 0, 0.099, 0.037500000000000006)	
> [4]	(0.14662485907553549, 0.22780757097791871, 0, 0.020000000000000004, 0.1375)	
> [5]	(0.20601042841037204, 0.25619873817034766, 0, 0.085, 0.17500000000000002)	
> [6]	(0.11857384441939123, 0.26992113564668774, 0, 0.090000000000000011, 0.1)	
> [7]	(0.30443912063134149, 0.22280757097791859, 0, 0.068, 0.1125)	
> [8]	(0.16507187147688843, 0.29952681388012614, 0, 0.06899999999999992, 0.075000000...	
> [9]	(0.10314261555806081, 0.3868296529968453, 0, 0.059, 0.037500000000000006)	
> [10]	(0.12599351747463358, 0.22654574132492145, 0, 0.063, 0.1)	
> [11]	(0.18120772266065388, 0.15932176656151431, 0, 0.078000000000000014, 0.125)	
> [12]	(0.30722237880496045, 0.23152996845425858, 0, 0.05699999999999995, 0.087500000...	
> [13]	(0.22417559188275082, 0.56063091482649829, 0, 0, 0.05)	
> [14]	(0.2381271138669673, 0.22638801261829719, 0, 0, 0.0125)	
> [15]	(0.2431440248027058, 0.1538801261829652, 0, 0.072, 0.087500000000000008)	
> [16]	(0.17325958286358509, 0.6, 0, 0.040000000000000008, 0.0625)	

Figura 21 – Punctele normalizate

Importanța normalizării este foarte mare în cadrul algoritmilor de machine learning, deoarece aceasta contribuie la obținerea unor rezultate mai stabile și mai corecte. Prin aducerea tuturor caracteristicilor la o scară comună, algoritmul poate analiza datele într-un mod echilibrat, fără ca anumite valori să influențeze disproporționat procesul de clusterizare. Astfel, se îmbunătățește precizia grupării obiectivelor turistice, iar rezultatele obținute devin mai coerente și mai relevante. În cadrul aplicației, normalizarea ajută algoritmul să compare corect obiectivele pe baza coordonatelor geografice, tipului, rating-ului și prețului estimativ, fiecare caracteristică având o contribuție controlată în procesul de analiză.

Fără această etapă, algoritmul ar produce clustere dezechilibrate și recomandări mai puțin precise, afectând experiența utilizatorului și calitatea generală a sistemului de recomandare implementat în aplicația TravelS.

3.5.3 Determinarea numărului optim de clustere

După pregătirea și normalizarea datelor, următoarea etapă importantă în cadrul algoritmului K-Means este determinarea numărului optim de clustere. Această etapă are un rol esențial deoarece performanța și acuratețea sistemului de recomandare depind în mod direct de modul în care obiectivele turistice sunt grupate.

În algoritmul K-Means, valoarea K reprezintă numărul de clustere în care vor fi împărțite datele. Alegerea unei valori prea mici poate conduce la gruparea unor obiective foarte diferite în același cluster, ceea ce reduce relevanța recomandărilor. Pe de altă parte, utilizarea unui număr foarte mare de clustere poate produce grupuri prea mici și fragmentate, afectând capacitatea sistemului de a identifica relații relevante între obiectivele turistice.

Din acest motiv, alegerea valorii optime pentru K reprezintă una dintre cele mai importante probleme în procesul de clusterizare.

Această problemă este rezolvată utilizând metoda Elbow („metoda cotului”), una dintre cele mai utilizate tehnici pentru evaluarea numărului optim de clustere în algoritmul K-Means.[11]

Principiul metodei constă în executarea algoritmului pentru mai multe valori ale lui K și calcularea erorii totale pentru fiecare variantă. Ulterior, valorile obținute sunt reprezentate grafic (Figura 22), iar punctul în care scăderea erorii începe să se stabilizeze indică numărul optim de clustere.

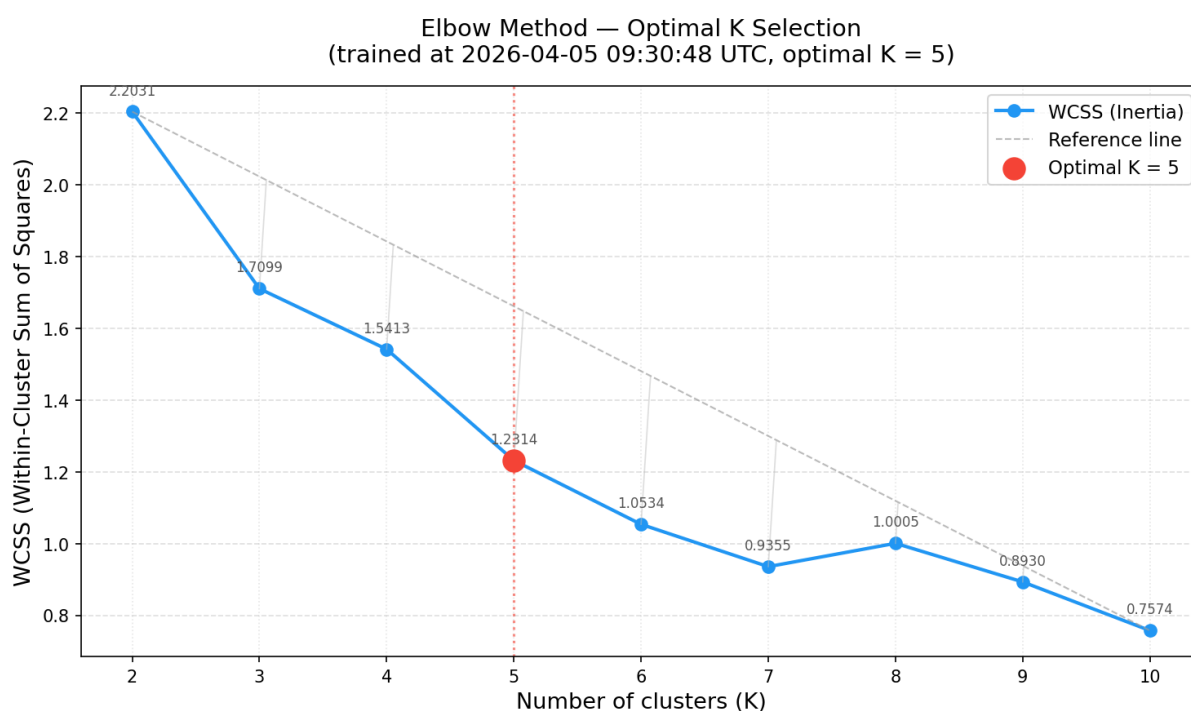


Figura 22 – Metoda Elbow pentru k [2:10]

Pentru măsurarea erorii este utilizată funcția WCSS (Within Cluster Sum of Squares), care calculează suma pătratelor distanțelor dintre fiecare punct și centroidul clusterului său.

Mai jos este reprezentată formula utilizată pentru calcularea valorii WCSS (Figura 23).

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

Figura 23 – Formula WCSS

unde:

- K reprezintă numărul de clustere;
- C_i reprezintă clusterul i;
- x reprezintă un punct din cluster;
- μ_i reprezintă centroidul clusterului.

Valoarea WCSS indică gradul de compactare al clusterelor. Cu cât valoarea este mai mică, cu atât punctele din același cluster sunt mai apropiate între ele.

În practică, algoritmul este executat pentru mai multe valori ale lui K. Pentru fiecare valoare este calculată valoarea WCSS, iar rezultatele sunt reprezentate grafic.

În fazele inițiale, creșterea numărului de clustere produce o reducere semnificativă a erorii. Totuși, după un anumit punct, îmbunătățirile devin din ce în ce mai mici. Acest punct de schimbare poartă denumirea de „elbow point” și indică valoarea optimă pentru K.

Conceptual, metoda urmărește identificarea unui echilibru între compactarea clusterelor și complexitatea modelului.

Un număr prea mare de clustere ar conduce la supraîmpărțirea datelor, în timp ce un număr prea mic ar produce grupuri foarte generale și mai puțin relevante.

Metoda Elbow este utilizată automat înaintea procesului de clusterizare propriu-zis. Sistemul analizează mai multe variante și selectează automat valoarea optimă pentru K, ceea ce permite adaptarea algoritmului la dimensiunea și distribuția actuală a datelor.

Un avantaj important al acestei abordări este flexibilitatea. Pe măsură ce numărul obiectivelor turistice din platformă crește, algoritmul poate recalcula automat structura clusterelor și poate adapta numărul optim de grupuri fără intervenție manuală.

Pentru analiză și monitorizare, valorile calculate sunt salvate într-un fișier de tip JSON, care conține valorile testate pentru K, valorile WCSS obținute pentru fiecare configurație analizată și numărul optim de clustere selectat în urma aplicării metodei.

Aceste informații pot fi utilizate ulterior pentru vizualizarea procesului de clusterizare și pentru analiza performanței algoritmului.

```
{"timestamp":"2026-05-23",  
"optimalK":6,"  
dataPoints":[  
{"k":2,"wcss":2.2033187067276523},  
{"k":3,"wcss":1.710718133334895},  
{"k":4,"wcss":1.4498831347479153},  
{"k":5,"wcss":1.3231878190131112},  
{"k":6,"wcss":1.054159871362918},  
{"k":7,"wcss":0.8939556624845718}  
]}
```

Determinarea corectă a valorii lui K are un impact direct asupra calității recomandărilor generate de sistem. Un model bine calibrat permite gruparea obiectivelor turistice în clustere coerente și contribuie la identificarea unor relații relevante între locații.

Prin utilizarea metodei Elbow, aplicația TravelS poate determina automat o structură optimă a clusterelor, oferind astfel un fundament solid pentru etapele următoare ale algoritmului de recomandare.

3.4.4 Aplicarea algoritmului K-Means și atribuirea clusterelor

După determinarea numărului optim de clustere, procesul de analiză continuă cu aplicarea efectivă a algoritmului K-Means asupra setului de obiective turistice normalizate.

Valoarea K stabilită prin metoda Elbow indică numărul de grupuri în care vor fi împărțite datele, însă atribuirea concretă a fiecărui obiectiv unui cluster este realizată în etapa următoare.

Algoritmul primește ca date de intrare vectorii normalizați aferenți obiectivelor turistice și numărul optim de clustere determinat anterior. Fiecare vector conține informații privind localizarea geografică, rating-ul mediu, prețul și tipul obiectivului. Deoarece datele au fost deja normalizate și ponderate, distanțele calculate în această etapă reflectă importanța relativă stabilită pentru fiecare caracteristică.

Aplicarea algoritmului începe prin alegerea unor centroizi inițiali. Un centroid reprezintă punctul central al unui cluster și are rolul de a caracteriza grupul de obiective asociat acestuia. În implementarea realizată, centroizii inițiali sunt selectați dintre punctele existente în setul de date. Odată stabilite aceste puncte inițiale, fiecare obiectiv turistic este comparat cu toți centroizii existenți și este atribuit clusterului al cărui centroid se află la cea mai mică distanță.

Într-un spațiu multidimensional, distanța dintre un obiectiv turistic și centroidul clusterului este calculată cu ajutorul distanței euclidiene, reprezentată în Figura 24. Aceasta distanță ține cont de cele cinci caracteristici utilizate pentru analiză:

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (r_2 - r_1)^2 + (pr_2 - pr_1)^2 + (t_2 - t_1)^2}$$

Figura 24 – Distanța Euclidiană

unde:

- x și y reprezintă coordonatele geografice normalizate;
- r reprezintă rating-ul mediu normalizat;
- pr reprezintă prețul normalizat;
- t reprezintă tipul obiectivului, exprimat numeric și normalizat.

Prin această formulă, fiecare obiectiv este comparat cu centroizii disponibili, iar clusterul atribuit este cel pentru care distanța calculată este minimă. Astfel, două obiective apropiate geografic, cu rating-uri, prețuri și categorii asemănătoare, au o probabilitate mai mare de a fi incluse în același grup.

În implementare, aplicarea algoritmului este inițiată după determinarea numărului optim de clustere, prin apelarea metodei care primește datele normalizate și valoarea K:

```

var normalizedPoints = NormalizePoints(points);
int optimalK = FindOptimalK(normalizedPoints);
var clusters = ApplyKMeans(normalizedPoints, optimalK);

```

Prima instrucțiune obține valorile normalizate ale caracteristicilor. A doua determină numărul optim de clustere pe baza metodei Elbow. În final, metoda **ApplyKMeans** efectuează atribuirea propriu-zisă a fiecărui obiectiv unui cluster.

În interiorul algoritmului, fiecare punct este comparat cu centroizii disponibili:

```

foreach (var point in points)
{
    var minDistance = double.MaxValue;
    var cluster = 0;

    for (int i = 0; i < k; i++)
    {
        var distance = CalculateDistance(point, centroids[i]);
        if (distance < minDistance)
        {
            minDistance = distance;
            cluster = i;
        }
    }
    clusters.Add(cluster);
}

```

Fragmentul de cod evidențiază etapa de atribuire a obiectivelor. Pentru fiecare punct este calculată distanța față de fiecare centroid, iar identificatorul clusterului cu distanța minimă este salvat în lista rezultată.

După atribuirea inițială, centroizii sunt recalculați cu ajutorul formulei prezente în Figura 25. Noul centroid al unui cluster este determinat prin media caracteristicilor tuturor obiectivelor incluse în acel cluster:

$$\mu_i = \frac{1}{|C_i|} \sum_{p \in C_i} p$$

Figura 25 – Formula de calcul a noilor centroizi

unde:

- μ_i reprezintă centroidul clusterului i ;
- C_i reprezintă mulțimea obiectivelor aparținând clusterului;

- |Ci| reprezintă numărul de obiective din cluster.

În aplicație, recalcularea centroizilor se realizează pentru fiecare dimensiune utilizată în procesul de clusterizare:

```
for (int i = 0; i < k; i++)
{
    var clusterPoints = points.Where((p, index) => clusters[index] == i).ToList();
    if (clusterPoints.Any())
    {
        centroids[i] = (
            clusterPoints.Average(p => p.x),
            clusterPoints.Average(p => p.y),
            clusterPoints.Average(p => p.rating),
            clusterPoints.Average(p => p.price),
            clusterPoints.Average(p => p.type)
        );
    }
}
```

Procesul de atribuire și recalculare este repetat până când obiectivele nu își mai schimbă clusterul sau până la atingerea unui număr maxim de iterații. Această condiție este necesară pentru a evita executarea nelimitată a algoritmului în situații în care datele nu conduc rapid la o structură stabilă.

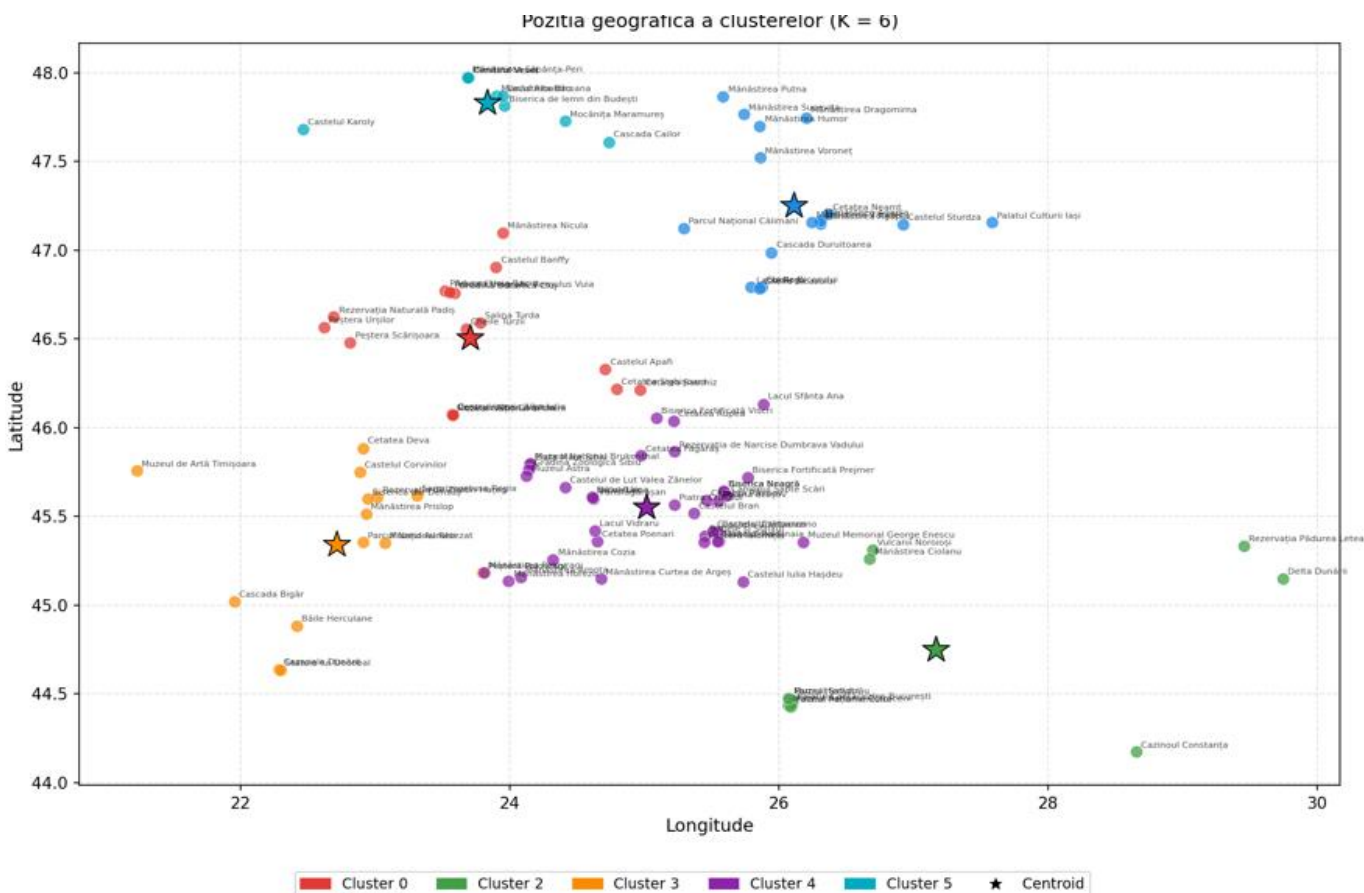


Figura 26 – Reprezentarea clusterelor in functie de lat & long

În urma finalizării acestei etape, fiecare obiectiv turistic este asociat unui cluster așa cum reiese din Figura 26. În termeni funcționali, clusterul nu reprezintă o categorie fixă definită manual, ci un grup identificat automat pe baza caracteristicilor datelor. Astfel, același cluster poate conține obiective care prezintă asemănări relevante din punctul de vedere al locației, prețului, evaluării și tipului.

3.4.5 Persistența rezultatelor clusterizării

După obținerea clusterelor, rezultatele trebuie păstrate astfel încât să poată fi utilizate ulterior pentru generarea recomandărilor și pentru analiza funcționării algoritmului. În cadrul aplicației TravelS, rezultatele procesului de clusterizare sunt salvate atât în baza de date, cât și în fișiere de tip JSON.

Primul rezultat salvat este identificatorul clusterului asociat fiecărui obiectiv. Entitatea utilizată pentru reprezentarea obiectivelor conține proprietatea **ClusterId**, în care este memorată apartenența obiectivului la grupul determinat de algoritm. După finalizarea procesului K-Means, identificatorii generați sunt atribuiți obiectivelor și salvați în baza de date:

```
for (int i = 0; i < objectives.Count; i++)
{
    objectives[i].ClusterId = clusters[i];
}
```

```
await _context.SaveChangesAsync();
```

Prin salvarea proprietății **ClusterId**, informația rezultată în urma clusterizării devine disponibilă și pentru cererile ulterioare. Astfel, în momentul în care un utilizator accesează

	Name text	ClusterId integer
1	Muzeul Național Brukenthal	4
2	Sarmizegetusa Regia	3
3	Cetatea Deva	3
4	Poiana Brașov	4
5	Mănăstirea Curtea de Argeș	4
6	Mănăstirea Sănătaș	5

Figura 27 – Atribuirea numărului de cluster pentru fiecare obiectiv turistic

pagina unui obiectiv, aplicația poate identifica imediat clusterul acestuia, fără a executa din nou întregul algoritm.

Pe lângă identificatorii atribuiți fiecărui obiectiv, sistemul salvează și centroizii clusterelor într-o structură dedicată.

Conceptual, un centroid salvat descrie profilul mediu al obiectivelor dintr-un cluster. Acesta este util în etapa de recomandare deoarece permite identificarea rapidă a clusterelor apropiate de obiectivul selectat.

Exemplul de cod utilizat pentru construirea centroizilor este următorul:

```
centroids.Add(new ClusterCentroid
{
    ClusterId = group.Key,
    CentroidX = group.Average(g => g.Point.X),
    CentroidY = group.Average(g => g.Point.Y),
    CentroidRating = group.Average(g => g.Point.AverageRating),
    CentroidPrice = group.Average(g => g.Point.Price),
    CentroidType = group.Average(g => (double)g.Point.TypeId),
    ObjectiveCount = group.Count(),
    LastUpdated = DateTime.UtcNow
});
```

Înainte de salvarea noilor centroizi, valorile existente sunt eliminate, deoarece o reantrenare a algoritmului poate modifica gruparea obiectivelor. În acest mod, baza de date păstrează doar structura actuală a clusterelor.

```
_context.ClusterNeighbors.RemoveRange(_context.ClusterNeighbors);
await _context.SaveChangesAsync();

await _context.ClusterNeighbors.AddRangeAsync(neighbors);
await _context.SaveChangesAsync();
```

Această strategie asigură consistența dintre clusterelor atribuite obiectivelor și centroizii utilizați ulterior pentru recomandări.

În plus, în implementare este utilizat un mecanism simplu de evitare a recalculării inutile a modelului. Dacă procesul de clusterizare a fost deja executat, iar numărul obiectivelor nu s-a modificat, aplicația poate evita repetarea antrenării:

```
if (_isTrained && _lastTrainedObjectiveCount == objectives.Count)
{
    _logger.LogInformation("ML model already trained for {Count} objectives, skipping.",
objectives.Count);
    return;
```



```
}
```

Acest mecanism reduce timpul de procesare în situațiile în care datele analizate au rămas neschimbate. Totuși, ca direcție ulterioară de îmbunătățire, verificarea poate fi extinsă astfel încât să detecteze și modificări ale rating-urilor, prețurilor, locațiilor sau tipurilor de obiective, nu doar schimbarea numărului total de înregistrări.

3.4.6 Generarea recomandărilor pe baza clusterelor obținute

După salvarea rezultatelor clusterizării, sistemul poate utiliza informațiile obținute pentru a genera recomandări. Această etapă este cea care transformă rezultatul matematic al algoritmului într-o funcționalitate vizibilă pentru utilizator.

Recomandările sunt generate în momentul în care utilizatorul vizualizează detaliile unui obiectiv turistic. Sistemul pornește de la obiectivul curent și identifică alte obiective care prezintă caracteristici asemănătoare. Recomandările nu sunt selectate din întreaga bază de date fără restricții, ci sunt căutate mai întâi în clusterul obiectivului curent și în clusterelor apropiate acestuia.

Prima operație constă în identificarea obiectivului pentru care trebuie generate recomandările. Pentru ca procesul să poată continua, obiectivul trebuie să existe, să aibă un cluster atribuit și să conțină o locație geografică validă.

```
var objective = await _context.Objectives.FirstOrDefaultAsync(o => o.Id == objectiveId);
```

```
if (objective == null || !objective.ClusterId.HasValue || objective.Location == null)  
    return result;
```

După identificarea obiectivului, sunt selectate clusterelor apropiate. Această operație utilizează centroizii salvați anterior și calculează proximitatea dintre poziția obiectivului curent și pozițiile medii ale clusterelor. Sunt selectate clusterul propriu și un număr restrâns de cluster vecine.

```
var nearestClusterIds = centroids  
    .OrderBy(c => Math.Pow(c.CentroidX - objX, 2) + Math.Pow(c.CentroidY - objY, 2))  
    .Take(1 + neighborCount) // clusterul propriu + vecini  
    .Select(c => c.ClusterId)  
    .ToHashSet();  
  
nearestClusterIds.Add(objective.ClusterId!.Value);
```

Această abordare are două avantaje importante. În primul rând, reduce numărul de obiective comparate, ceea ce îmbunătățește performanța. În al doilea rând, permite includerea unor obiective relevante din clustere apropiate, evitând limitarea recomandărilor exclusiv la grupul obiectivului curent.

După stabilirea clusterelor relevante, sunt extrase obiectivele candidate. Obiectivul curent este eliminat din lista candidaților, deoarece nu trebuie recomandat utilizatorului un element pe care acesta îl vizualizează deja. În plus, sunt încărcate imaginile și recenziile asociate obiectivelor, deoarece aceste informații trebuie afișate ulterior în interfață.

```
var candidateObjectives = await _context.Objectives
    .Include(o => o.Reviews)
    .Include(o => o.Images)
    .Where(o => nearbyClusters.Contains(o.ClusterId!.Value) && o.Id != objectiveId &&
o.Location != null)
    .ToListAsync();
```

Pentru fiecare candidat este calculat un scor de similaritate. Spre deosebire de procesul de clusterizare, care urmărește identificarea unor grupuri generale, scorul de similaritate permite ordonarea fină a obiectivelor recomandate pentru un obiectiv concret.

Scorul final poate fi exprimat prin formula:

$$S(o_1, o_2) = w_{loc} \cdot S_{loc} + w_{rating} \cdot S_{rating} + w_{price} \cdot S_{price} + w_{type} \cdot S_{type}$$

Figura 28 – Formula scorului final

unde:

- $S(o_1, o_2)$ reprezintă scorul de similaritate dintre două obiective;
- S_{loc} reprezintă similaritatea din punctul de vedere al locației;
- S_{rating} reprezintă apropierea rating-urilor;
- S_{price} reprezintă apropierea valorilor de preț;
- S_{type} reprezintă asemănarea tipului obiectivului;
- w reprezintă ponderea atribuită fiecărui criteriu.

În implementarea realizată, componenta geografică are ponderea principală, deoarece proximitatea constituie un criteriu important în alegerea unor noi obiective turistice. Tipul

obiectivului contribuie, de asemenea, la păstrarea unei similitudini tematice, în timp ce rating-ul și prețul rafinează rezultatele.

Calculul scorului este realizat astfel:

```
var locationScore = 1.0 - (obj1.Location!.Distance(obj2.Location!) * 111000 / 100000);
var ratingScore = obj1.Reviews.Any() && obj2.Reviews.Any()
    ? 1.0 - Math.Abs(obj1.Reviews.Average(r => r.Raiting) - obj2.Reviews.Average(r =>
r.Raiting)) / 5.0
    : 0.5;
var priceScore = !string.IsNullOrEmpty(obj1.Pret) && !string.IsNullOrEmpty(obj2.Pret)
    ? 1.0 - Math.Abs(double.Parse(obj1.Pret) - double.Parse(obj2.Pret)) / 1000.0
    : 0.5;
var typeScore = obj1.Type == obj2.Type ? 1.0 : 0.0;

return (locationScore * _locationWeight) + (ratingScore * _ratingWeight) + (priceScore *
_priceWeight) + (typeScore * _typeWeight);
```

Distanța geografică este calculată pornind de la proprietățile spațiale ale obiectivelor. Valoarea rezultată este convertită aproximativ în metri prin multiplicarea cu 111000, corespunzător aproximării gradelor geografice în distanță liniară. Într-o versiune ulterioară, această operație poate fi realizată direct prin funcții geospațiale PostGIS proiectate pentru măsurarea distanțelor reale.

După calcularea scorurilor, obiectivele sunt sortate mai întâi descrescător după similaritate și apoi crescător după distanță. În acest mod, aplicația prioritizează obiectivele care au un profil apropiat de cel vizualizat, iar în cazul unor scoruri asemănătoare afișează cu prioritate locațiile mai apropiate.

```
var recommendations = candidateObjectives
    .Select(o => new
    {
        Objective = o,
        Similarity = CalculateSimilarityScore(objective, o)
    })
    .OrderByDescending(x => x.Similarity)
    .Take(count)
```

Rezultatul returnat către interfața utilizatorului conține informațiile necesare afișării recomandărilor: identificatorul obiectivului, denumirea, orașul, prima imagine disponibilă, rating-ul mediu și distanța față de obiectivul curent (Figura 29).

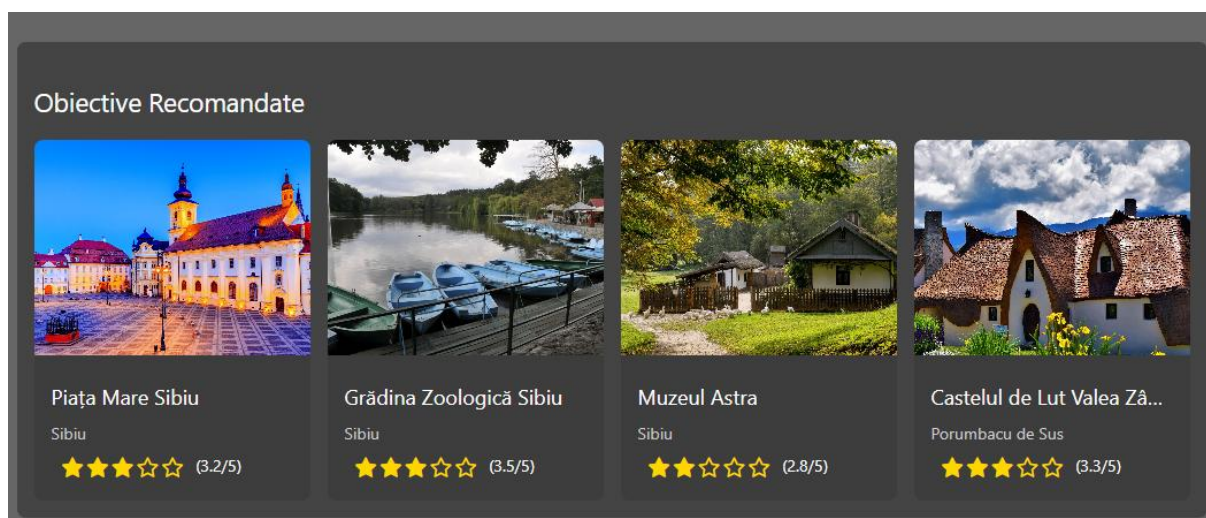


Figura 29 – Afișarea obiectivelor recomandate în interfață

Sistemul include și posibilitatea salvării relațiilor dintre obiectivele recomandate într-o tabelă dedicată. Pentru fiecare obiectiv sursă pot fi memorate obiectivele țintă, distanța și scorul de similaritate asociat. Această abordare permite reutilizarea recomandărilor calculate anterior și poate reduce timpul de răspuns la afișarea paginii de detaliu.

În concluzie, după determinarea numărului optim de clustere, algoritmul nu se oprește la gruparea datelor, ci utilizează clusterelor obținute ca mecanism de filtrare inițială a candidaților. Recomandarea finală este realizată printr-un scor ponderat de similaritate, care ia în calcul proximitatea geografică, rating-ul, prețul și tipul obiectivului. Această combinație permite aplicației TravelS să furnizeze recomandări relevante, explicabile și adaptate specificului platformei turistice.

3.6 Proiectarea ChatBot-ului

Integrarea unei componente conversaționale în cadrul aplicației a avut ca obiectiv facilitarea accesului utilizatorilor la informațiile disponibile despre obiective turistice și utilizarea mai naturală a platformei. Într-o aplicație turistică, utilizatorul poate avea nevoie de informații punctuale, precum obiectivele care pot fi vizitate într-o anumită localitate, detalii despre o locație sau recomandări generale referitoare la planificarea unei vizite. În lipsa unui mecanism conversațional, obținerea acestor informații presupune navigarea manuală prin paginile aplicației și consultarea succesivă a mai multor elemente de conținut.

Pentru reducerea acestui efort a fost dezvoltat un chatbot inteligent, capabil să primească întrebări formulate în limbaj natural și să genereze răspunsuri pe baza informațiilor disponibile în sistem. Spre deosebire de un chatbot bazat exclusiv pe răspunsuri prestabilite, componenta realizată în aplicația TravelS utilizează documente turistice indexate semantic și un model lingvistic local, astfel încât răspunsurile să poată fi formulate într-un mod natural și adaptat întrebării introduse de utilizator.

Arhitectura chatbotului este construită în jurul conceptului de **Retrieval-Augmented Generation** (RAG)[12]. În cadrul acestei abordări, modelul lingvistic nu este utilizat izolat, ci primește înainte de generarea răspunsului un context extras dintr-o bază de date. Aceastameste formată din documente referitoare la informațiile turistice, iar căutarea relevantă este realizată printr-o bază vectorială FAISS[13]. Astfel, răspunsul nu este generat numai pe baza cunoștințelor generale ale modelului, ci este fundamentat pe informațiile identificate ca fiind apropiate semantic de întrebarea utilizatorului (Figura 30).

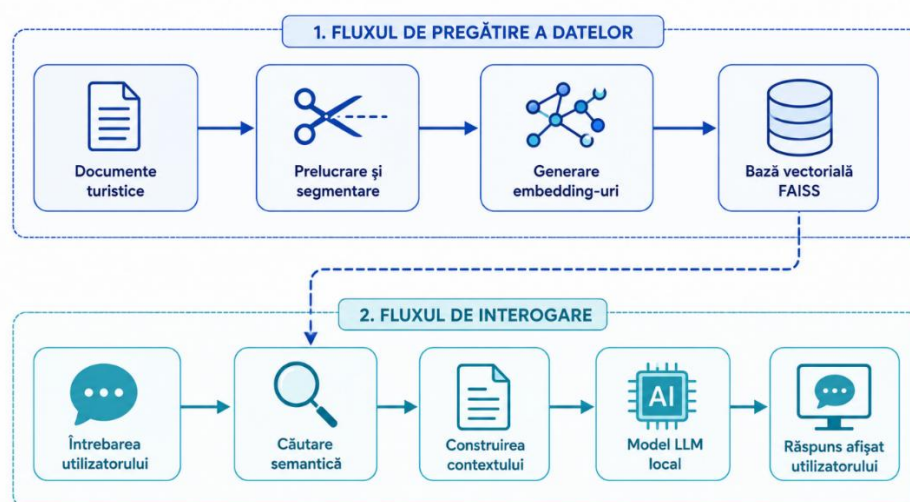


Figura 30 – Fluxul general al chatbot-ului în aplicație

3.6.1 Construirea bazei de date vectoriale

Prima etapă necesară funcționării chatbotului a constat în pregătirea informațiilor care urmau să fie utilizate pentru formularea răspunsurilor. În locul introducerii manuale a unor răspunsuri fixe, aplicația permite procesarea mai multor tipuri de documente, astfel încât baza de cunoștințe să poată fi extinsă gradual cu informații noi.

Documentele sunt preluate dintr-un director local dedicat. Pentru a permite utilizarea unor surse variate, sistemul acceptă mai multe formate de fișiere, printre care fișiere text, documente PDF, documente Word, fișiere CSV, pagini HTML și fișiere Markdown. Pentru fiecare extensie este utilizat un loader corespunzător furnizat de ecosistemul LangChain[14].

Fragmentul conceptual prin care sunt asociate tipurile de fișiere cu mecanismele de încărcare este următorul:

```
loader_map = {
    ".txt": TextLoader,
    ".pdf": PDFPlumberLoader,
    ".docx": UnstructuredWordDocumentLoader,
    ".csv": CSVLoader,
    ".html": UnstructuredHTMLLoader,
    ".md": UnstructuredMarkdownLoader
}
```

Prin această abordare, baza de cunoștințe nu este dependentă de un singur format de intrare. Documentele pot proveni din surse diferite, iar informațiile disponibile pot fi actualizate prin simpla adăugare a unor fișiere noi în directorul destinat procesării.

În prima fază, sistemul parcurge recursiv fișierele disponibile, prezente în Figura 31. Fiecare document valid este încărcat și adăugat într-o colecție comună. În cazul apariției unor probleme la procesarea unui fișier, eroarea este semnalată, însă procesarea celorlalte documente poate continua. Această abordare oferă robustețe procesului de construire a bazei de date vectoriale.

	13 Locuri de Vizitat Langa Sibiu - Atractii...	07.06.2026 21:26	Chrome HTML Do...
	AboutTravelS.docx	24.03.2026 21:22	Microsoft Word D...
	Licenta.docx	24.03.2026 21:22	Microsoft Word D...
	Obiective.csv	24.03.2026 21:22	Excel.CSV
	obiective-turistice-pdf.pdf	07.06.2026 21:24	Microsoft Edge PD...

Figura 31 – Exemple de documente folosite

3.6.2 Segmentarea documentelor

După încărcarea documentelor, informațiile nu sunt introduse integral în baza vectorială, ci sunt împărțite în fragmente mai mici, denumite *chunk-uri*. Această etapă este importantă deoarece un document complet poate conține informații despre mai multe subiecte, iar identificarea unui pasaj relevant devine mai dificilă dacă întreaga sursă este tratată ca o singură unitate.

În aplicația TravelS, segmentarea este realizată utilizând **RecursiveCharacterTextSplitter**, configurat cu o dimensiune maximă de 500 de caractere pentru fiecare fragment și o suprapunere de 100 de caractere între fragmente consecutive:

```
text_splitter = RecursiveCharacterTextSplitter(  
    chunk_size=500,  
    chunk_overlap=100  
)  
  
chunks = text_splitter.split_documents(documents)
```

Alegerea unei suprapuneri între fragmente are rolul de a evita pierderea contextului la granița dintre două segmente. De exemplu, dacă descrierea unui obiectiv turistic începe într-un fragment și continuă în următorul, suprapunerea contribuie la păstrarea legăturii semantice dintre informații.

În mod conceptual, un document este transformat într-o serie de fragmente care îl definesc. Dimensiunea fragmentelor influențează direct calitatea căutării semantice. Fragmentele foarte mari pot conține informații nerelevante pentru întrebarea utilizatorului, în timp ce fragmentele foarte mici pot pierde contextul necesar formulării unui răspuns coerent. Configurația aleasă urmărește un echilibru între precizia căutării și păstrarea semnificației textului.

3.6.3 Generarea embedding-urilor și construirea bazei vectoriale

După segmentarea documentelor, fiecare fragment este transformat într-o reprezentare numerică numită *embedding*. Un embedding reprezintă un vector care surprinde semnificația semantică a textului, astfel încât fragmente cu sens asemănător să poată fi identificate chiar dacă nu conțin exact aceleași cuvinte.

Pentru generarea embedding-urilor a fost utilizat modelul:

```
HuggingFaceEmbeddings(  
    model_name="intfloat/multilingual-e5-large"  
)
```

Alegerea modelului *intfloat/multilingual-e5-large* este justificată de faptul că aplicația este destinată utilizatorilor care formulează întrebări în limba română, iar modelul oferă suport multilingv. Astfel, sistemul poate compara semantic întrebări și informații redactate în limba română, fără a depinde exclusiv de coincidențe lexicale.

De exemplu, întrebările „Ce obiective pot vizita în Sibiu?” și „Ce locuri turistice sunt disponibile în Sibiu?” au formulări diferite, însă exprimă o intenție foarte apropiată. Prin reprezentarea sub formă de embedding-uri, cele două întrebări pot fi asociate cu fragmente similare din documentele turistice.

Embedding-urile generate sunt stocate utilizând FAISS, o bibliotecă specializată în indexarea și căutarea eficientă în colecții de vectori. La prima rulare, baza vectorială este creată pornind de la fragmentele procesate:

```
vectorstore = FAISS.from_documents(chunks, embeddings)
```

Dacă baza vectorială există deja, aceasta este încărcată, iar noile documente sunt adăugate fără a fi necesară reconstruirea completă a structurii:

```
vectorstore = FAISS.load_local(  
    db_path,  
    embeddings,  
    allow_dangerous_deserialization=True  
)
```

```
vectorstore.add_documents(chunks)
```

La final, baza vectorială actualizată este salvată local în directorul `vector_db`:

```
vectorstore.save_local(db_path)
```

Această soluție oferă posibilitatea extinderii bazei de date pe parcursul dezvoltării aplicației. Astfel, noi documente turistice pot fi introduse și indexate ulterior, fără modificarea logicii chatbotului.

3.6.4 Căutarea semantică a informațiilor relevante

După construirea bazei vectoriale, chatbotul poate primi întrebări din partea utilizatorului. La fiecare solicitare, întrebarea este transformată într-un vector compatibil cu vectorii existenți în FAISS, iar sistemul identifică cele mai apropiate fragmente din punct de vedere semantic.

În implementarea realizată, sunt extrase primele patru fragmente considerate cele mai relevante:

```
docs_with_scores = self.vectorstore.similarity_search_with_score(
    query,
    k=4
)
```

Rezultatul căutării este alcătuit din chunk-urile identificate și scorurile asociate acestora. Scorurile sunt utilizate pentru a decide dacă informațiile locale sunt suficient de relevante pentru formularea răspunsului. În configurația utilizată, a fost stabilit un prag de relevanță:

RELEVANCE_THRESHOLD = 1.2

Exemplu de chunk-uri trimise modelului:

Istoricul conversației:

Assistant: Bună! Sunt asistentul TravelS. Cu ce te pot ajuta?

User: Ce este TravelS?

Context:

Document 1:

Descriere platformă TravelS

TravelS este o platformă web modernă, dedicată turismului din România, care le oferă utilizatorilor o experiență completă de explorare, planificare și împărtășire a locurilor vizitate. Aplicația permite fiecărui utilizator să își creeze un cont, să se autentifice și să își administreze datele personale într-un mod securizat și intuitiv.

Metadata: {"source": "documente\\Descriere_TravelS_Demo_Chatbot.docx"}

Document 2:

Descriere platformă TravelS

TravelS este o platformă web modernă, dedicată turismului din România, care le oferă utilizatorilor o experiență completă de explorare, planificare și împărtășire a locurilor vizitate. Aplicația permite fiecărui utilizator să își creeze un cont, să se autentifice și să își administreze datele personale într-un mod securizat și intuitiv.

Metadata: {"source": "documente\\Descriere_TravelS_Demo_Chatbot.docx"}

Document 3:

Descriere platformă TravelS

TravelS este o platformă web modernă, dedicată turismului din România, care le oferă utilizatorilor o experiență completă de explorare, planificare și împărtășire a locurilor vizitate. Aplicația permite fiecărui utilizator să își creeze un cont, să se autentifice și să își administreze datele personale într-un mod securizat și intuitiv.

Metadata: {"source": "documente\\AboutTravelS.docx"}

Document 4:

TravelS se poziționează în domeniul aplicațiilor web dedicate ghidajului turistic digital, răspunzând nevoii tot mai evidente de a integra tehnologia în promovarea și accesarea informațiilor de interes cultural, istoric sau recreativ. Această platformă se adresează în principal turiștilor care intenționează să viziteze orașe și regiuni din România și care își doresc o soluție rapidă și eficientă pentru a afla ce pot face sau vedea într-un anumit loc. Fie că este vorba despre un vizitator

Metadata: {"source": "documente\\Licenta.docx"}

În cazul căutării FAISS utilizate în implementare, un scor mai mic indică, în mod uzual, o apropiere mai mare între vectorul întrebării și vectorul documentului. Astfel, dacă sunt găsite fragmente cu scoruri sub pragul stabilit, sistemul utilizează documentele locale drept context pentru generarea răspunsului. Dacă nu este identificat niciun document relevant sau dacă toate scorurile depășesc pragul, chatbotul recurge la o sursă alternativă de informații.

3.6.5 Construirea promptului și generarea răspunsului

După identificarea contextului relevant, informațiile sunt organizate într-un prompt transmis modelului lingvistic. Promptul reprezintă instrucțiunea completă pe baza căreia modelul va genera răspunsul și conține trei elemente principale:

- istoricul conversației;
- contextul extras din documente sau din căutarea web;
- întrebarea curentă a utilizatorului.

Pentru răspunsurile bazate pe documentele locale, a fost definit un șablon prin care modelului i se solicită să utilizeze doar contextul disponibil și să indice situația în care informația cerută nu este prezentă:

"Folosește contextul de mai jos pentru a răspunde la întrebare.\n"

"Dacă informația nu se găsește în context, răspunde cu "

"Nu am informații despre asta.\n"

Această regulă contribuie la reducerea riscului ca modelul să genereze informații care nu sunt susținute de documentele disponibile.

Un alt element important îl reprezintă istoricul conversației. Acesta este inclus în prompt pentru a permite modelului să păstreze continuitatea dialogului. Astfel, utilizatorul poate formula întrebări succesive care depind de răspunsurile anterioare, fără a repeta de fiecare dată întregul context.

Request primit:

```
{
  "message": "Ce obiective pot vizita în Sibiu?",
  "chat_history": [
    {
      "role": "assistant",
      "content": "Bună! Sunt asistentul TravelS. Cu ce te pot ajuta?"
    },
    {
      "role": "user",
      "content": "Ce este TravelS?"
    },
    {
      "role": "assistant",
      "content": "TravelS este o platformă web modernă dedicată turismului din România, care oferă utilizatorilor o experiență completă de explorare, planificare și împărtășire a locurilor vizitate. Aplicația permite fiecărui utilizator să creeze un cont, să se autentifice și să administreze datele personale într-un mod securizat și intuitiv. Se adresează în principal turiștilor care intenționează să viziteze orașe și regiuni din România, oferind informații despre activități culturale, istorice sau recreative disponibile în diferite locații."
    }
  ]
}
```

}

3.6.6 Integrarea modelului lingvistic local prin LM Studio

Pentru generarea răspunsurilor a fost utilizat un model lingvistic rulat local prin intermediul LM Studio, ilustrat în Figura 32. Această aplicație permite încărcarea și rularea modelelor LLM pe calculatorul propriu și expune o interfață compatibilă cu API-ul OpenAI.

În cadrul implementării, conexiunea către modelul local este realizată utilizând următoarea configurare:

```
self.llm = ChatOpenAI(  
    openai_api_base="http://localhost:1234/v1",  
    openai_api_key="lm-studio",  
    temperature=0.3,  
    model="microsoft/phi-4"  
)
```

Modelul utilizat este *microsoft/phi-4*, iar parametrul *temperature* este configurat la valoarea 0.3. O valoare redusă a acestui parametru determină generarea unor răspunsuri mai stabile și mai puțin creative, aspect util într-o aplicație informativă, unde claritatea și fidelitatea față de context sunt mai importante decât varietatea formulărilor.

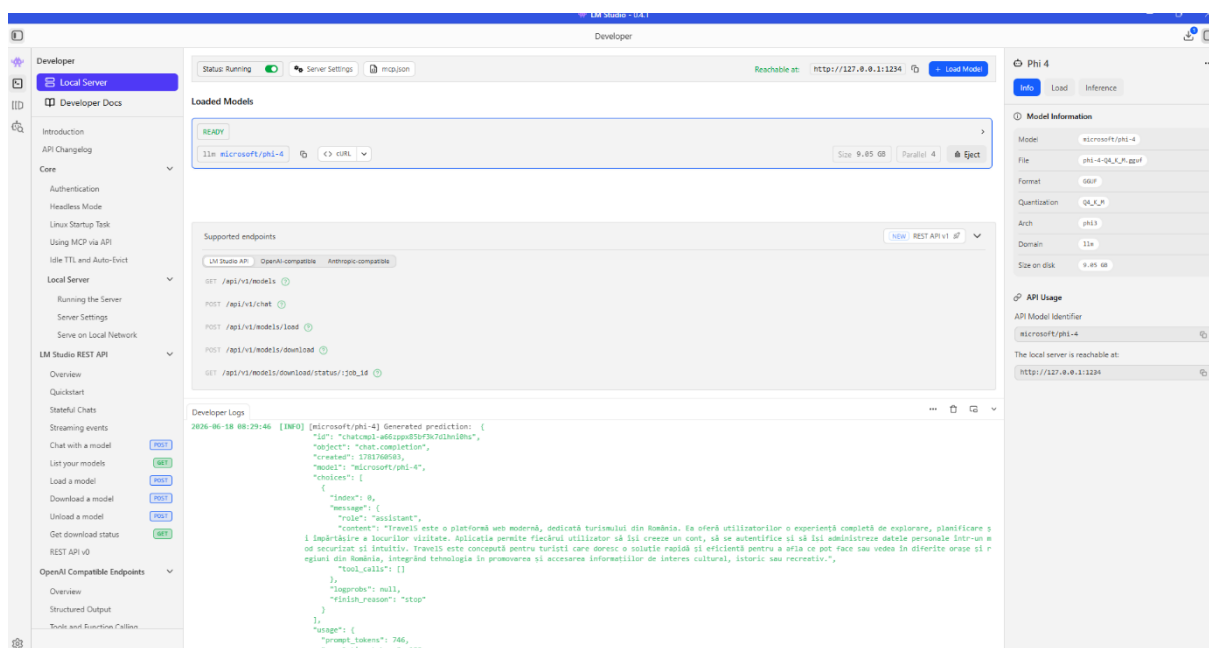


Figura 32 – LM Studio cu Phi-4 expus prin API

Utilizarea unui model local prezintă mai multe avantaje. În primul rând, reduce dependența față de servicii externe comerciale. În al doilea rând, permite controlul asupra modului de procesare a datelor și evită transmiterea automată a întrebărilor utilizatorilor către servicii cloud. În plus, oferă flexibilitate în testarea și schimbarea modelelor utilizate.

După construirea promptului, acesta este transmis modelului, iar răspunsul generat este returnat componentei care a inițiat cererea:

```
response = self.llm.invoke([HumanMessage(content=prompt)])
answer = response.content
```

Rezultatul final include nu doar textul răspunsului, ci și promptul utilizat, contextul extras și sursa informației. Aceste date sunt utile pentru testare și verificarea modului în care chatbotul a formulat răspunsul.

3.6.7 Expunerea chatbotului printr-un serviciu API

Pentru ca interfața aplicației să poată comunica cu chatbotul, funcționalitatea acestuia a fost expusă prin intermediul unui serviciu web dezvoltat în Python, utilizând framework-ul Flask. Aplicația definește endpoint-ul `/api/chat`, care primește mesajul utilizatorului și istoricul conversației printr-o cerere de tip *POST*.

Structura generală a cererii este următoarea:

```
{
  „message”: „Ce obiective pot vizita în Sibiu?”,
  „chat_history”: []
}
```

La primirea cererii, serviciul verifică existența mesajului, obține instanța chatbotului și apelează metoda responsabilă de generarea răspunsului:

```
data = request.get_json()

message = data[„message”]
chat_history = data.get(„chat_history”, [])

chatbot = get_chatbot()
response_data = chatbot.get_response(message, chat_history)
```

Răspunsul returnat frontend-ului conține mai multe informații utile:

```
{
  „response”: „Răspunsul generat pentru utilizator”,
  „logs”: „Informații despre procesare”,
  „prompt”: „Promptul transmis modelului”,
  „context”: „Contextul identificat”,
  „source”: „documents”
}
```

Pe lângă răspunsul efectiv, integrarea Socket.IO permite transmiterea în timp real a informațiilor despre procesul de generare, precum log-urile, promptul, contextul și sursa utilizată. Această funcționalitate este deosebit de utilă în etapa de dezvoltare și testare, deoarece permite observarea modului în care răspunsul a fost construit.

4. Concluzii și dezvoltări ulterioare

Dezvoltarea aplicației TravelS a urmărit realizarea unei platforme web dedicate descoperirii și promovării obiectivelor turistice, evenimentelor și experiențelor din România. Proiectul a pornit de la necesitatea centralizării informațiilor turistice într-o aplicație accesibilă, interactivă și adaptată cerințelor utilizatorilor moderni. În cadrul soluției realizate au fost integrate tehnologii web moderne, funcționalități bazate pe localizare, un sistem de recomandare și o componentă conversațională bazată pe inteligență artificială.

Aplicația dezvoltată demonstrează posibilitatea combinării unei arhitecturi web clasice, alcătuite din frontend, backend și bază de date, cu funcționalități moderne precum procesarea datelor geospațiale, gruparea automată a obiectivelor turistice și căutarea semantică cu informații din documente. Prin această abordare, TravelS oferă utilizatorilor un mod mai simplu și mai intuitiv de a descoperi locații de interes și de a accesa informații utile pentru planificarea unei vizite.

4.1 Obiectivele îndeplinite

Principalul obiectiv al lucrării a fost dezvoltarea unei aplicații web funcționale, capabile să gestioneze și să prezinte informații turistice într-un mod organizat și ușor de utilizat. Acest

obiectiv a fost atins prin implementarea unei interfețe grafice dezvoltate în Vue.js și a unui backend realizat în C# ASP.NET Core Web API, cele două componente comunicând prin intermediul unor servicii REST.

În cadrul aplicației au fost implementate funcționalități pentru afișarea și gestionarea obiectivelor turistice, a evenimentelor, a experiențelor și a itinerariilor. Pentru administrarea conținutului a fost realizat un modul dedicat, prin intermediul căruia pot fi efectuate operații de adăugare, modificare, vizualizare și ștergere asupra entităților principale ale sistemului.

Un alt obiectiv important a fost gestionarea informațiilor geografice asociate locațiilor turistice. Utilizarea bazei de date PostgreSQL împreună cu extensia PostGIS a permis stocarea coordonatelor și realizarea unor operații bazate pe distanță și poziție geografică. Această componentă este relevantă pentru o aplicație turistică, deoarece locația reprezintă unul dintre criteriile principale în procesul de alegere a unui obiectiv.

În ceea ce privește securitatea, aplicația include un mecanism de autentificare bazat pe token-uri JWT de tip Bearer. Acesta permite identificarea utilizatorilor autentificați și protejarea funcționalităților care nu trebuie să fie accesibile publicului larg, în special cele specifice modulului de administrare.

O funcționalitate importantă realizată în cadrul proiectului este sistemul de recomandare a obiectivelor turistice. Prin utilizarea algoritmului K-Means, obiectivele sunt grupate în funcție de caracteristici precum locația, tipul, rating-ul și prețul. Pe baza acestor grupări, aplicația poate propune utilizatorului obiective asemănătoare celui vizualizat, contribuind la o navigare mai fluidă și la descoperirea mai rapidă a unor locații relevante.

De asemenea, integrarea chatbotului a adăugat aplicației o componentă inteligentă de interacțiune. Prin utilizarea embedding-urilor, a bazei vectoriale FAISS și a unui model lingvistic local rulat prin LM Studio, utilizatorul poate adresa întrebări în limbaj natural și poate primi răspunsuri construite pe baza informațiilor turistice procesate în sistem.

Prin implementarea acestor funcționalități, aplicația TravelS îndeplinește obiectivele principale propuse la începutul lucrării și oferă o bază solidă pentru dezvoltări viitoare.

4.2 Posibile îmbunătățiri și dezvoltări ulterioare

Deși aplicația TravelS oferă funcționalitățile principale necesare unei platforme turistice digitale, aceasta poate fi extinsă și îmbunătățită în mai multe direcții. O primă direcție importantă vizează sistemul de recomandare. În forma actuală, recomandările sunt generate pe baza caracteristicilor obiectivelor turistice, precum locația geografică, tipul, prețul și rating-ul. În viitor, algoritmul ar putea lua în considerare și comportamentul utilizatorilor, de exemplu obiectivele vizualizate, locațiile salvate, rating-urile acordate sau itinerariile create.

Prin introducerea acestor informații, sistemul ar putea evolua de la recomandări bazate predominant pe similaritatea obiectivelor la un model hibrid, care să combine caracteristicile locațiilor cu preferințele reale ale utilizatorilor. De asemenea, algoritmul K-Means poate fi îmbunătățit prin utilizarea unei metode mai stabile de inițializare a centroizilor, precum K-Means++ [15], prin analizarea obiectivelor atipice care pot influența gruparea și prin introducerea unor metode suplimentare de evaluare a calității clusterelor.

O altă direcție de dezvoltare privește extinderea componentei chatbot. Baza de cunoștințe poate fi îmbogățită cu documente suplimentare despre obiective turistice, evenimente, trasee și informații locale. Totodată, răspunsurile generate ar putea include sursele utilizate, astfel încât utilizatorul să poată verifica informațiile primite. Integrarea unor surse oficiale actualizate ar îmbunătăți calitatea și credibilitatea răspunsurilor oferite de sistem.

Aplicația poate fi dezvoltată și prin introducerea unui mecanism mai avansat de creare automată a itinerariilor. Pe baza locației utilizatorului, a timpului disponibil, a distanțelor dintre obiective și a preferințelor acestuia, sistemul ar putea propune un traseu complet de vizitare pentru o zi sau pentru o perioadă mai lungă. O astfel de funcționalitate ar crește utilitatea practică a platformei în etapa de planificare a unei excursii.

În ceea ce privește conținutul aplicației, o dezvoltare ulterioară ar putea permite implicarea mai activă a utilizatorilor prin propunerea de noi obiective, încărcarea de imagini și experiențe personale sau raportarea informațiilor incorecte. Pentru menținerea calității conținutului, aceste contribuții ar trebui asociate unui mecanism de validare și aprobare de către administratori.

Din perspectiva disponibilității aplicației, platforma ar putea fi publicată într-un mediu cloud și optimizată pentru accesarea de pe dispozitive mobile. Transformarea acesteia într-o aplicație de tip Progressive Web Application[16] sau dezvoltarea unei aplicații mobile dedicate

ar facilita utilizarea directă în timpul călătoriilor. În paralel, securitatea poate fi îmbunătățită prin stocarea securizată a datelor de configurare, utilizarea exclusivă a comunicațiilor HTTPS și introducerea unor mecanisme suplimentare pentru gestionarea sesiunilor și protejarea datelor utilizatorilor.

Prin aceste îmbunătățiri, TravelS ar putea evolua de la o platformă funcțională de prezentare și recomandare a obiectivelor turistice către un asistent digital complex, capabil să sprijine utilizatorul pe întreg parcursul planificării și desfășurării unei călătorii.

4.3 Concluzii

Realizarea aplicației TravelS a permis valorificarea cunoștințelor dobândite în domeniul dezvoltării aplicațiilor web, al bazelor de date, al securității și al inteligenței artificiale. Proiectul a presupus proiectarea și integrarea mai multor componente distincte, de la interfața grafică și serviciile backend până la gestionarea datelor geospațiale, generarea recomandărilor și implementarea chatbotului.

Rezultatul obținut este o aplicație web care răspunde unei nevoi reale: accesarea rapidă și organizată a informațiilor turistice și descoperirea unor obiective relevante într-un mod interactiv. Utilizarea tehnologiilor Vue.js, ASP.NET Core, PostgreSQL, PostGIS, FAISS și LM Studio a permis construirea unei soluții moderne și extensibile, în care funcționalitățile clasice ale unei platforme turistice sunt completate de mecanisme inteligente de recomandare și interacțiune conversațională.

În concluzie, TravelS reprezintă atât rezultatul practic al procesului de dezvoltare desfășurat în cadrul lucrării, cât și un punct de plecare pentru o platformă care poate fi îmbunătățită și extinsă ulterior. Funcționalitățile implementate demonstrează aplicabilitatea tehnologiilor moderne în domeniul turismului digital și evidențiază potențialul utilizării inteligenței artificiale și al datelor geospațiale pentru oferirea unor experiențe turistice mai accesibile, mai relevante și mai personalizate.

Anexa 1

User Story	Prioritate	Complexitate
Ca utilizator, vreau să caut obiective turistice după nume sau categorie, astfel încât să găsesc rapid locațiile de interes.	Ridică	Ușoară
Ca utilizator, vreau să filtrez obiectivele turistice în funcție de categorie, oraș sau rating, pentru a vizualiza doar rezultatele relevante.	Ridică	Ușoară
Ca utilizator, vreau să vizualizez obiectivele turistice pe hartă, astfel încât să înțeleg mai bine localizarea acestora.	Ridică	Medie
Ca utilizator, vreau să văd ruta către un obiectiv turistic folosind Google Maps, pentru a ajunge mai ușor la destinație.	Ridică	Medie
Ca utilizator, vreau să creez și să gestionez itinerarii personalizate, astfel încât să îmi planific eficient călătoria.	Ridică	Medie
Ca utilizator, vreau să vizualizez evenimente locale și informațiile aferente acestora, pentru a participa la activități relevante.	Medie	Medie
Ca utilizator, vreau să vizualizez imagini și videoclipuri pentru fiecare obiectiv turistic, pentru a obține o prezentare mai realistă a locației.	Ridică	Medie
Ca utilizator, vreau să adaug recenzii și evaluări pentru obiectivele turistice, astfel încât să împărtășesc experiența mea.	Medie	Medie
Ca utilizator, vreau să marchez obiective favorite, astfel încât să le pot accesa rapid ulterior.	Medie	Ușoară
Ca utilizator, vreau să primesc recomandări personalizate de obiective turistice, pentru a descoperi locații similare intereselor mele.	Ridică	Dificilă
Ca utilizator, vreau să folosesc un chatbot inteligent pentru a obține rapid informații despre obiective și destinații turistice.	Medie	Dificilă
Ca administrator, vreau să gestionez obiectivele turistice prin operații CRUD, astfel încât să mențin platforma actualizată.	Ridică	Medie
Ca administrator, vreau să aprob sau să resping sugestiile utilizatorilor pentru obiective și evenimente, pentru a controla calitatea conținutului.	Medie	Medie

Sursa imaginilor:

Figura 9 - <https://www.cs.cmu.edu/~dst/WordEmbeddingDemo/figures/fig1.png>

Bibliografie

- [1] Buhalis, D. (2003). *eTourism: Information technology for strategic tourism management*. Pearson education.
- [2] Xiang, Z., & Gretzel, U. (2010). Role of social media in online travel information search. *Tourism management*, 31(2), 179-188.
- [3] Shapira, B., Rokach, L., & Ricci, F. (2022). Recommender systems handbook.
- [4] Bulchand-Gidumal, J. (2022). Impact of artificial intelligence in travel, tourism, and hospitality. In *Handbook of e-Tourism* (pp. 1943-1962). Cham: Springer International Publishing.
- [5] Lucassen, G., Dalpiaz, F., van der Werf, J. M. E., & Brinkkemper, S. (2016). Improving agile requirements: the quality user story framework and tool. *Requirements engineering*, 21(3), 383-403.
- [6] Fink, G., & Flatow, I. (2014). Introducing single page applications. In *Pro single page application development: Using backbone.js and asp.net* (pp. 3-13). Berkeley, CA: Apress.
- [7] Garaguso, P. D. (2023). *Vue.js 3 Design Patterns and Best Practices: Develop scalable and robust applications with Vite, Pinia, and Vue Router*. Packt Publishing Ltd.
- [8] ASP ,NET Core in .NET https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis?view=aspnetcore-10.0&WT.mc_id=dotnet-35129-website
- [9] Gu, S. (2024). A survey of large language models in tourism (Tourism LLMs). *Preprint on Qeios*.
- [10] NPSQL - .NET Access to PostgreSQL <https://www.npgsql.org/>
- [11] Cui, M. (2020). Introduction to the k-means clustering algorithm based on the elbow method. *Accounting, Auditing and Finance*, 1(1), 5-8.
- [12] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 32.
- [13] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P. E., ... & Jégou, H. (2025). The faiss library. *IEEE Transactions on Big Data*.
- [14] Kotiyal, A., Gujjar, P., Prasad, G., Devadas, R. M., Hiremani, V., & Tangade, P. (2024, July). Chat with PDF using LangChain model. In *2024 Second international conference on advances in information technology (ICAIT)* (Vol. 1, pp. 1-4). IEEE.
- [15] Bahmani, B., Moseley, B., Vattani, A., Kumar, R., & Vassilvitskii, S. (2012). Scalable k-means++. *arXiv preprint arXiv:1203.6402*.

- [16] Tandel, S., & Jamadar, A. (2018). Impact of progressive web apps on web app development. *International Journal of Innovative Research in Science, Engineering and Technology*, 7(9), 9439-9444.
- [17] Cuomo, S. (2024). *Vue.js 3 for Beginners: Learn the essentials of Vue.js 3 and its ecosystem to build modern web applications*. Packt Publishing Ltd. 7
- [18] Smith, J. P. (2021). *Entity Framework core in action*. Simon and Schuster.
- [19] Rana, M., Pandey, A., Mishra, A., & Kandu, V. (2017). Enhancing data security: a comprehensive study on the efficacy of JSON web token (JWT) and HMAC SHA-256 algorithm for web application security. *signature*, 11, 12.
- [20] Garaguso, P. D. (2023). *Vue.js 3 Design Patterns and Best Practices: Develop scalable and robust applications with Vite, Pinia, and Vue Router*. Packt Publishing Ltd.